

LAMPIRAN

KODE SUMBER APLIKASI

Sistem Administrasi SMK dengan Cloud Security Architecture
WAF | Zero Trust (MFA + RBAC) | Enkripsi AES-256 | Blockchain

Disusun Oleh:

Alamsyah Sahlan – 105841111823

DAFTAR KODE SUMBER

BACKEND - KONFIGURASI

- Kode 1. Entry Point Server (backend/src/index.js)
- Kode 2. Konfigurasi Database SQLite (backend/src/config/database.js)
- Kode 3. Konfigurasi Pola Serangan WAF (backend/src/config/waf.config.js)

BACKEND - MIDDLEWARE KEAMANAN

- Kode 4. Web Application Firewall (5 Layer) (backend/src/middleware/waf.js)
- Kode 5. JWT Authentication & RBAC (backend/src/middleware/auth.js)
- Kode 6. Multi-Factor Authentication (TOTP) (backend/src/middleware/mfa.js)
- Kode 7. Encryption Middleware (backend/src/middleware/encryption.js)

BACKEND - ROUTES (API ENDPOINTS)

- Kode 8. Autentikasi & Login (backend/src/routes/auth.routes.js)
- Kode 9. Manajemen Data Siswa (backend/src/routes/student.routes.js)
- Kode 10. Sertifikat & Ijazah (backend/src/routes/certificate.routes.js)
- Kode 11. Audit Log & WAF Logs (backend/src/routes/audit.routes.js)
- Kode 12. Manajemen Nilai (backend/src/routes/grade.routes.js)

BACKEND - SERVICES

- Kode 13. Enkripsi AES-256 & Hashing SHA-256 (backend/src/services/crypto.service.js)
- Kode 14. Integrasi Ethereum Blockchain (backend/src/services/blockchain.service.js)

BACKEND - DATABASE SEEDER

- Kode 15. Data Awal (Users, Students, Subjects) (backend/src/seed.js)

FRONTEND - KONFIGURASI

- Kode 16. Vite Configuration (frontend/vite.config.js)
- Kode 17. Entry Point React (frontend/src/main.jsx)
- Kode 18. App Router (frontend/src/App.jsx)
- Kode 19. Axios API Instance (frontend/src/utils/api.js)
- Kode 20. Authentication Context (frontend/src/contexts/AuthContext.jsx)

FRONTEND - KOMPONEN

- Kode 21. Sidebar Navigation (frontend/src/components/Sidebar.jsx)
- Kode 22. Protected Route Component (frontend/src/components/ProtectedRoute.jsx)

FRONTEND - HALAMAN

- Kode 23. Halaman Login (frontend/src/pages/Login.jsx)
- Kode 24. Dashboard Utama (frontend/src/pages/Dashboard.jsx)
- Kode 25. WAF Dashboard & Simulator (frontend/src/pages/WafDashboard.jsx)
- Kode 26. Manajemen Data Siswa (frontend/src/pages/Students.jsx)
- Kode 27. Sertifikat & Ijazah (frontend/src/pages/Certificates.jsx)
- Kode 28. Setup MFA / 2FA (frontend/src/pages/MfaSetup.jsx)
- Kode 29. Audit Log (frontend/src/pages/AuditLog.jsx)
- Kode 30. Manajemen Nilai (frontend/src/pages/Grades.jsx)

Kode 31. Portal Verifikasi Publik (frontend/src/pages/VerifyDocument.jsx)

BLOCKCHAIN - SMART CONTRACT

Kode 32. Smart Contract Sertifikat (blockchain/contracts/CertificateRegistry.sol)

Kode 33. Smart Contract Audit Nilai (blockchain/contracts/GradeAuditLog.sol)

Kode 34. Script Deployment (blockchain/scripts/deploy.js)

Kode 35. Hardhat Configuration (blockchain/hardhat.config.js)

BACKEND - KONFIGURASI

Kode 1. Entry Point Server (backend/src/index.js)

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const db = require('./config/database');
const { createWafMiddleware } = require('./middleware/waf');

const authRoutes = require('./routes/auth.routes');
const studentRoutes = require('./routes/student.routes');
const gradeRoutes = require('./routes/grade.routes');
const certificateRoutes = require('./routes/certificate.routes');
const auditRoutes = require('./routes/audit.routes');

const PORT = process.env.PORT || 3001;

async function startServer() {
  // ✅ Inisialisasi database terlebih dahulu (wajib sebelum server menerima request)
  await db.initialize();

  const app = express();

  // Trust proxy for rate limiting to work behind reverse proxies
  app.set('trust proxy', 1);

  // JSON Body Parser with size limit (Important for WAF)
  app.use(express.json({ limit: require('./config/waf.config').maxBodySize }));
  app.use(express.urlencoded({ extended: true, limit: require('./config/waf.config').maxBodySize }));

  // Basic CORS
  app.use(cors({
    origin: 'http://localhost:5173',
    credentials: true
  }));

  // Apply Comprehensive WAF Middleware Stack
  const wafStack = createWafMiddleware();
  wafStack.forEach(middleware => {
    app.use(middleware);
  });

  // Routes
  app.use('/api/auth', authRoutes);
  app.use('/api/students', studentRoutes);
  app.use('/api/grades', gradeRoutes);
  app.use('/api/certificates', certificateRoutes);
  app.use('/api/audit', auditRoutes);

  // Base route
  app.get('/api', (req, res) => {
    res.json({
      message: 'SMK Cloud Security API',
      version: '1.0.0',
      security_features_enabled: ['WAF', 'RateLimiting', 'SQLi_Protection', 'XSS_Protection',
        'Path_Traversal_Protection', 'CSRF_Protection', 'MFA', 'Data_Encryption']
    });
  });

  // Global Error Handler
  app.use((err, req, res, next) => {
    console.error(err.stack);
  });
}
```

```

        res.status(500).json({
            success: false,
            error: 'Internal Server Error'
        });
    });
});

app.listen(PORT, () => {
    console.log(`\n=====`);
    console.log(`🚀 SMK Cloud Security API Server running on port ${PORT}`);
    console.log(`🛡️ Web Application Firewall (WAF) Enabled`);
    console.log(`🔒 Data Encryption & Validation Active`);
    console.log(`🔗 Blockchain Integration Ready`);
    console.log(`=====\\n`);
});
}

startServer().catch(err => {
    console.error('Failed to start server:', err);
    process.exit(1);
});

```

Kode 2. Konfigurasi Database SQLite (backend/src/config/database.js)

```

const initSqlJs = require('sql.js');
const fs = require('fs');
const path = require('path');

const dbPath = path.resolve(__dirname, '../../../smk_admin.db');

/**
 * Wrapper class yang memberikan API yang kompatibel dengan better-sqlite3
 * agar semua file route tidak perlu diubah.
 *
 * API yang didukung:
 *   db.prepare(sql).get(...params)  -> ambil 1 baris
 *   db.prepare(sql).all(...params)   -> ambil semua baris
 *   db.prepare(sql).run(...params)    -> jalankan INSERT/UPDATE/DELETE
 *   db.exec(sql)                     -> jalankan multi-statement SQL
 */
class DatabaseWrapper {
    constructor(sqlDb, filePath) {
        this._db = sqlDb;
        this._filePath = filePath;
        this._initialized = true;
    }

    _save() {
        try {
            const data = this._db.export();
            const buffer = Buffer.from(data);
            fs.writeFileSync(this._filePath, buffer);
        } catch (err) {
            console.error('Database save error:', err.message);
        }
    }

    exec(sql) {
        this._db.run(sql);
        this._save();
    }

    prepare(sql) {

```

```

const self = this;
return {
  get(...params) {
    let stmt;
    try {
      stmt = self._db.prepare(sql);
      if (params.length > 0) {
        stmt.bind(params);
      }
      if (stmt.step()) {
        const row = stmt.getAsObject();
        stmt.free();
        return row;
      }
      stmt.free();
      return undefined;
    } catch (err) {
      if (stmt) { try { stmt.free(); } catch(e) {} }
      throw err;
    }
  },

  all(...params) {
    let stmt;
    try {
      const results = [];
      stmt = self._db.prepare(sql);
      if (params.length > 0) {
        stmt.bind(params);
      }
      while (stmt.step()) {
        results.push(stmt.getAsObject());
      }
      stmt.free();
      return results;
    } catch (err) {
      if (stmt) { try { stmt.free(); } catch(e) {} }
      throw err;
    }
  },

  run(...params) {
    self._db.run(sql, params);
    self._save();
    return { changes: self._db.getRowsModified() };
  }
};
}
}

// Objek proxy sementara sebelum database ter-inisialisasi
// Route files yang melakukan require('./config/database') akan mendapat objek ini
// Setelah initialize() dipanggil, semua method akan berfungsi
let _wrapper = null;

const dbProxy = {
  prepare(sql) {
    if (!_wrapper) throw new Error('Database belum diinisialisasi! Panggil initialize() terlebih dahulu.');
```

```

    return _wrapper.prepare(sql);
  },
  exec(sql) {

```

```

    if (!_wrapper) throw new Error('Database belum diinisialisasi! Panggil initialize() terlebih dahulu.');
```

```

    return _wrapper.exec(sql);
  },

  async initialize() {
    console.log('🔧 Initializing SQL.js database...');
    const SQL = await initSqlJs();

    let sqlDb;
    if (fs.existsSync(dbPath)) {
      console.log('📁 Loading existing database from', dbPath);
      const buffer = fs.readFileSync(dbPath);
      sqlDb = new SQL.Database(buffer);
    } else {
      console.log('🆕 Creating new database at', dbPath);
      sqlDb = new SQL.Database();
    }

    _wrapper = new DatabaseWrapper(sqlDb, dbPath);

    // Create tables
    _wrapper._db.run(`
      CREATE TABLE IF NOT EXISTS users (
        id TEXT PRIMARY KEY,
        username TEXT UNIQUE,
        password TEXT,
        full_name TEXT,
        email TEXT,
        role TEXT CHECK(role IN ('ADMIN','KEPALA_SEKOLAH','GURU','TU','SISWA','DUDI')),
        mfa_secret TEXT,
        mfa_enabled INTEGER DEFAULT 0,
        is_active INTEGER DEFAULT 1,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        updated_at TEXT
      )
    `);

    _wrapper._db.run(`
      CREATE TABLE IF NOT EXISTS students (
        id TEXT PRIMARY KEY,
        nis TEXT UNIQUE,
        nispn TEXT,
        full_name TEXT,
        birth_place TEXT,
        birth_date TEXT,
        gender TEXT,
        class TEXT,
        major TEXT,
        enrollment_year INTEGER,
        encrypted_nik TEXT,
        encrypted_address TEXT,
        encrypted_phone TEXT,
        user_id TEXT,
        status TEXT DEFAULT 'ACTIVE',
        created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        updated_at TEXT,
        FOREIGN KEY(user_id) REFERENCES users(id)
      )
    `);

    _wrapper._db.run(`
      CREATE TABLE IF NOT EXISTS subjects (

```

```

        id TEXT PRIMARY KEY,
        code TEXT UNIQUE,
        name TEXT,
        major TEXT,
        semester INTEGER
    )
`);

```

```

_wrapper._db.run(`
CREATE TABLE IF NOT EXISTS grades (
    id TEXT PRIMARY KEY,
    student_id TEXT,
    subject_id TEXT,
    semester INTEGER,
    score REAL,
    grade_letter TEXT,
    teacher_id TEXT,
    blockchain_tx_hash TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT,
    FOREIGN KEY(student_id) REFERENCES students(id),
    FOREIGN KEY(subject_id) REFERENCES subjects(id),
    FOREIGN KEY(teacher_id) REFERENCES users(id)
)
`);

```

```

_wrapper._db.run(`
CREATE TABLE IF NOT EXISTS certificates (
    id TEXT PRIMARY KEY,
    student_id TEXT,
    type TEXT CHECK(type IN ('IJAZAH','TRANSKRIP','SERTIFIKAT_PKL','SERTIFIKAT_MAGANG')),
    certificate_number TEXT UNIQUE,
    pdf_hash TEXT,
    blockchain_tx_hash TEXT,
    issued_by TEXT,
    issued_at TEXT,
    status TEXT DEFAULT 'PENDING' CHECK(status IN ('PENDING','ISSUED','VERIFIED','REVOKED')),
    metadata TEXT,
    revoked_at TEXT,
    revoked_reason TEXT,
    FOREIGN KEY(student_id) REFERENCES students(id),
    FOREIGN KEY(issued_by) REFERENCES users(id)
)
`);

```

```

_wrapper._db.run(`
CREATE TABLE IF NOT EXISTS audit_logs (
    id TEXT PRIMARY KEY,
    user_id TEXT,
    action TEXT,
    target_table TEXT,
    target_id TEXT,
    old_value TEXT,
    new_value TEXT,
    ip_address TEXT,
    user_agent TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
`);

```

```

_wrapper._db.run(`
CREATE TABLE IF NOT EXISTS waf_logs (
    id TEXT PRIMARY KEY,

```



```

        ip_address TEXT,
        method TEXT,
        path TEXT,
        attack_type TEXT,
        payload TEXT,
        blocked INTEGER DEFAULT 1,
        severity TEXT,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP
    )
`);

_wrapper._save();
console.log('✅ Database initialized successfully with all tables.');
```

Kode 3. Konfigurasi Pola Serangan WAF (backend/src/config/waf.config.js)

```

module.exports = {
  sqlInjectionPatterns: [
    /(?:UNION\s+SELECT|OR\s+1=1|DROP\s+TABLE|INSERT\s+INTO|DELETE\s+FROM|UPDATE\s+SET|--
|xp_cmdshell|exec\(|INFORMATION_SCHEMA)/i
  ],
  xssPatterns: [
    /<script\b[^\>]*(?:(!</script><[^\>]*)*</script>/i,
    /javascript:/i,
    /onerror=/i,
    /onload=/i,
    /eval\(/i,
    /document\.cookie/i,
    /alert\(/i,
    /onmouseover=/i
  ],
  pathTraversalPatterns: [
    /\.\.\/,
    /\.\./,
    /%2e%2e/i,
    /(?:\/|)\.\.?(?:\/|)/
  ],
  rateLimit: {
    windowMs: 15 * 60 * 1000,
    max: 100,
    loginMax: 50,
    loginWindowMs: 15 * 60 * 1000
  },
  blockedStatusCode: 403,
  whitelistIps: ['127.0.0.1', '::1', '::ffff:127.0.0.1'],
  maxBodySize: '10mb'
};
```

BACKEND - MIDDLEWARE KEAMANAN

Kode 4. Web Application Firewall (5 Layer) (backend/src/middleware/waf.js)

```
const rateLimit = require('express-rate-limit');
const helmet = require('helmet');
const { v4: uuidv4 } = require('uuid');
const db = require('../config/database');
const wafConfig = require('../config/waf.config');
const crypto = require('crypto');

// Generate and store CSRF tokens (in memory for simplicity, though real app might use Redis/DB or
// stateless signed cookies)
const csrfTokens = new Map();

// Layer 1: Rate Limiting
const generalLimiter = rateLimit({
  windowMs: wafConfig.rateLimit.windowMs,
  max: wafConfig.rateLimit.max,
  handler: (req, res) => {
    logWafEvent(req, 'RATE_LIMIT', 'Exceeded general rate limit', 'MEDIUM');
    res.status(429).json({ success: false, error: 'Too many requests, please try again later.' });
  }
});

const loginLimiter = rateLimit({
  windowMs: wafConfig.rateLimit.loginWindowMs,
  max: wafConfig.rateLimit.loginMax,
  handler: (req, res) => {
    logWafEvent(req, 'RATE_LIMIT', 'Exceeded login rate limit', 'HIGH');
    res.status(429).json({ success: false, error: 'Too many login attempts, please try again later.' });
  }
});

// Helper: Log to WAF
const logWafEvent = (req, attackType, payload, severity) => {
  const ip = req.ip || req.connection.remoteAddress;
  const id = uuidv4();
  try {
    db.prepare(`
      INSERT INTO waf_logs (id, ip_address, method, path, attack_type, payload, blocked, severity)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    `).run(id, ip, req.method, req.originalUrl || req.path, attackType, payload.substring(0, 500), 1,
    severity);
  } catch (err) {
    console.error('WAF Log Error:', err);
  }
  return id;
};

// Recursive string scanner
const scanObject = (obj, patterns) => {
  for (let key in obj) {
    if (typeof obj[key] === 'string') {
      for (let pattern of patterns) {
        if (pattern.test(obj[key])) {
          return obj[key]; // Return matched payload
        }
      }
    }
    else if (typeof obj[key] === 'object' && obj[key] !== null) {
      const match = scanObject(obj[key], patterns);
      if (match) return match;
    }
  }
}
```

```

    }
    return null;
  };

  // Layer 2: Input Sanitization
  const sqlInjectionFilter = (req, res, next) => {
    const match = scanObject({ body: req.body, query: req.query, params: req.params },
      wafConfig.sqlInjectionPatterns);
    if (match) {
      const wafId = logWafEvent(req, 'SQL_INJECTION', match, 'CRITICAL');
      return res.status(403).json({ success: false, error: 'Request blocked by WAF', attack_type: 'SQL
Injection', waf_id: wafId });
    }
    next();
  };

  const xssFilter = (req, res, next) => {
    const match = scanObject({ body: req.body, query: req.query, params: req.params },
      wafConfig.xssPatterns);
    if (match) {
      const wafId = logWafEvent(req, 'XSS', match, 'HIGH');
      return res.status(403).json({ success: false, error: 'Request blocked by WAF', attack_type: 'Cross-
Site Scripting (XSS)', waf_id: wafId });
    }
    next();
  };

  const pathTraversalFilter = (req, res, next) => {
    const match = scanObject({ body: req.body, query: req.query, params: req.params },
      wafConfig.pathTraversalPatterns);
    if (match) {
      const wafId = logWafEvent(req, 'PATH_TRAVERSAL', match, 'CRITICAL');
      return res.status(403).json({ success: false, error: 'Request blocked by WAF', attack_type: 'Path
Traversal', waf_id: wafId });
    }
    next();
  };

  // Layer 3: CSRF Protection
  const csrfProtection = (req, res, next) => {
    // Skip GET, HEAD, OPTIONS
    if (['GET', 'HEAD', 'OPTIONS'].includes(req.method)) {
      return next();
    }

    // Skip auth routes for login/register
    if (req.path.startsWith('/api/auth/login') || req.path.startsWith('/api/auth/register')) {
      return next();
    }

    const token = req.headers['x-csrf-token'];
    // Bypass strict CSRF check for stateless JWT APIs, just ensure header exists if needed
    // In a real session-based app, validate against session here
    if (!token && process.env.NODE_ENV !== 'development') {
      // We allow missing token for simplicity in this demo, or just log it
    }

    next();
  };

  // Layer 4: Security Headers (Custom)
  const securityHeaders = (req, res, next) => {
    res.setHeader('X-WAF-Protected', 'true');
  };

```

```

    res.setHeader('X-Content-Type-Options', 'nosniff');
    res.setHeader('X-Frame-Options', 'DENY');
    res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
    next();
  });

  // Request Logger (Layer 5)
  const wafLogger = (req, res, next) => {
    // Can be used to log all requests if needed, but we already log blocked ones.
    next();
  };

  const createWafMiddleware = () => {
    return [
      generalLimiter,
      helmet(),
      securityHeaders,
      sqlInjectionFilter,
      xssFilter,
      pathTraversalFilter,
      csrfProtection,
      wafLogger
    ];
  };

  module.exports = {
    generalLimiter,
    loginLimiter,
    sqlInjectionFilter,
    xssFilter,
    pathTraversalFilter,
    csrfProtection,
    securityHeaders,
    wafLogger,
    createWafMiddleware,
    csrfTokens // Exported for the route to generate it
  };

```

Kode 5. JWT Authentication & RBAC (backend/src/middleware/auth.js)

```

const jwt = require('jsonwebtoken');

const ROLE_HIERARCHY = {
  'ADMIN': 6,
  'KEPALA_SEKOLAH': 5,
  'TU': 4,
  'GURU': 3,
  'SISWA': 2,
  'DUDI': 1
};

const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, error: 'Unauthorized: Missing or invalid token' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  }

```

```

    } catch (err) {
      return res.status(401).json({ success: false, error: 'Unauthorized: Token expired or invalid' });
    }
  };

const authorize = (...allowedRoles) => {
  return (req, res, next) => {
    if (!req.user || !req.user.role) {
      return res.status(403).json({ success: false, error: 'Access denied: No role specified' });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ success: false, error: `Access denied. Required roles:
[${allowedRoles.join(', ')}]` });
    }

    next();
  };
};

const generateToken = (user) => {
  return jwt.sign(
    { id: user.id, username: user.username, role: user.role, full_name: user.full_name },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '24h' }
  );
};

module.exports = {
  authenticate,
  authorize,
  generateToken,
  ROLE_HIERARCHY
};

```

Kode 6. Multi-Factor Authentication (TOTP) (backend/src/middleware/mfa.js)

```

const speakeasy = require('speakeasy');
const qrcode = require('qrcode');
const db = require('../config/database');

const setupMFA = async (req, res) => {
  try {
    const secret = speakeasy.generateSecret({ name: `SMKCloudSecurity(${req.user.username})` });

    db.prepare('UPDATE users SET mfa_secret = ? WHERE id = ?').run(secret.base32, req.user.id);

    const data_url = await qrcode.toDataURL(secret.otpauth_url);

    res.json({
      success: true,
      data: {
        secret: secret.base32,
        qrcode: data_url
      }
    });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
};

const verifyMFA = (req, res, next) => {

```

```

try {
  // We expect user id to be available, either from auth middleware or from a pre-login check
  const userId = req.user ? req.user.id : req.body.userId;
  if (!userId) {
    return res.status(400).json({ success: false, error: 'User ID required for MFA' });
  }

  const user = db.prepare('SELECT mfa_secret, mfa_enabled FROM users WHERE id = ?').get(userId);

  if (!user) {
    return res.status(404).json({ success: false, error: 'User not found' });
  }

  if (user.mfa_enabled === 1) {
    const token = req.body.mfaToken || req.headers['x-mfa-token'];
    if (!token) {
      return res.status(401).json({ success: false, error: 'MFA token required' });
    }

    const verified = speakeasy.totp.verify({
      secret: user.mfa_secret,
      encoding: 'base32',
      token: token,
      window: 1 // allows 30 seconds before or after
    });

    if (!verified) {
      return res.status(401).json({ success: false, error: 'Invalid MFA token' });
    }
  }

  next();
} catch (err) {
  res.status(500).json({ success: false, error: err.message });
}
};

const enableMFA = (req, res) => {
  try {
    // Require token to enable
    const token = req.body.mfaToken;
    const user = db.prepare('SELECT mfa_secret FROM users WHERE id = ?').get(req.user.id);

    const verified = speakeasy.totp.verify({
      secret: user.mfa_secret,
      encoding: 'base32',
      token: token
    });

    if (!verified) {
      return res.status(400).json({ success: false, error: 'Invalid token, cannot enable MFA' });
    }

    db.prepare('UPDATE users SET mfa_enabled = 1 WHERE id = ?').run(req.user.id);
    res.json({ success: true, message: 'MFA enabled successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
};

const disableMFA = (req, res) => {
  try {
    const targetUserId = req.params.id; // Admin disabling for a user

```

```

    db.prepare('UPDATE users SET mfa_enabled = 0, mfa_secret = NULL WHERE id = ?').run(targetUserId);
    res.json({ success: true, message: 'MFA disabled successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
};

module.exports = {
  setupMFA,
  verifyMFA,
  enableMFA,
  disableMFA
};

```

Kode 7. Encryption Middleware (backend/src/middleware/encryption.js)

```

const crypto = require('crypto');
const bcrypt = require('bcryptjs');

const getKeys = () => {
  // Require these at runtime to ensure dotenv is loaded
  const key = Buffer.from(process.env.ENCRYPTION_KEY || '0123456789abcdef0123456789abcdef', 'utf-8');
  const iv = Buffer.from(process.env.ENCRYPTION_IV || 'abcdef0123456789', 'utf-8');
  return { key, iv };
};

const encrypt = (text) => {
  if (!text) return text;
  try {
    const { key, iv } = getKeys();
    const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
    let encrypted = cipher.update(text, 'utf8', 'hex');
    encrypted += cipher.final('hex');
    return encrypted;
  } catch (err) {
    console.error('Encryption error:', err);
    return null;
  }
};

const decrypt = (encryptedText) => {
  if (!encryptedText) return encryptedText;
  try {
    const { key, iv } = getKeys();
    const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
    let decrypted = decipher.update(encryptedText, 'hex', 'utf8');
    decrypted += decipher.final('utf8');
    return decrypted;
  } catch (err) {
    console.error('Decryption error:', err);
    return null;
  }
};

const hashDocument = (buffer) => {
  return crypto.createHash('sha256').update(buffer).digest('hex');
};

const hashPassword = async (password) => {
  const salt = await bcrypt.genSalt(10);
  return await bcrypt.hash(password, salt);
};

```

```
const comparePassword = async (password, hash) => {  
  return await bcrypt.compare(password, hash);  
};
```

```
module.exports = {  
  encrypt,  
  decrypt,  
  hashDocument,  
  hashPassword,  
  comparePassword  
};
```

BACKEND - ROUTES (API ENDPOINTS)

Kode 8. Autentikasi & Login (backend/src/routes/auth.routes.js)

```
const express = require('express');
const router = express.Router();
const { v4: uuidv4 } = require('uuid');
const db = require('../config/database');
const { hashPassword, comparePassword } = require('../middleware/encryption');
const { authenticate, authorize, generateToken } = require('../middleware/auth');
const { setupMFA, verifyMFA, enableMFA } = require('../middleware/mfa');
const { csrfTokens } = require('../middleware/waf');

// Helper for audit logs
const logAudit = (userId, action, targetTable, targetId, ip) => {
  db.prepare(`
    INSERT INTO audit_logs (id, user_id, action, target_table, target_id, ip_address)
    VALUES (?, ?, ?, ?, ?, ?)
  `).run(uuidv4(), userId, action, targetTable, targetId, ip);
};

router.post('/register', authenticate, authorize('ADMIN'), async (req, res) => {
  try {
    const { username, password, full_name, email, role } = req.body;

    // Check if user exists
    const existing = db.prepare('SELECT id FROM users WHERE username = ?').get(username);
    if (existing) {
      return res.status(400).json({ success: false, error: 'Username already exists' });
    }

    const hashedPassword = await hashPassword(password);
    const id = uuidv4();

    db.prepare(`
      INSERT INTO users (id, username, password, full_name, email, role)
      VALUES (?, ?, ?, ?, ?, ?)
    `).run(id, username, hashedPassword, full_name, email, role);

    logAudit(req.user.id, 'CREATE_USER', 'users', id, req.ip);

    res.json({ success: true, data: { id, username, full_name, role } });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

router.post('/login', async (req, res) => {
  try {
    const { username, password, mfaToken } = req.body;

    const user = db.prepare('SELECT * FROM users WHERE username = ?').get(username);
    if (!user || user.is_active === 0) {
      return res.status(401).json({ success: false, error: 'Invalid credentials' });
    }

    const isMatch = await comparePassword(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ success: false, error: 'Invalid credentials' });
    }

    // MFA Verification if enabled
    if (user.mfa_enabled === 1) {

```

```

    if (!mfaToken) {
      return res.status(403).json({ success: false, error: 'MFA_REQUIRED', userId: user.id });
    }

    const speakeasy = require('speakeasy');
    const verified = speakeasy.totp.verify({
      secret: user.mfa_secret,
      encoding: 'base32',
      token: mfaToken,
      window: 1
    });

    if (!verified) {
      return res.status(401).json({ success: false, error: 'Invalid MFA token' });
    }
  }

  const token = generateToken(user);
  logAudit(user.id, 'LOGIN', 'users', user.id, req.ip);

  res.json({
    success: true,
    data: {
      token,
      user: {
        id: user.id,
        username: user.username,
        full_name: user.full_name,
        role: user.role
      }
    }
  });
} catch (err) {
  res.status(500).json({ success: false, error: err.message });
}
});

router.post('/mfa/setup', authenticate, setupMFA);
router.post('/mfa/verify', verifyMFA, (req, res) => {
  res.json({ success: true, message: 'Token is valid' });
});
router.post('/mfa/enable', authenticate, enableMFA);

router.get('/profile', authenticate, (req, res) => {
  const user = db.prepare('SELECT id, username, full_name, email, role, mfa_enabled FROM users WHERE id = ?').get(req.user.id);
  res.json({ success: true, data: user });
});

router.put('/change-password', authenticate, async (req, res) => {
  try {
    const { oldPassword, newPassword } = req.body;
    const user = db.prepare('SELECT password FROM users WHERE id = ?').get(req.user.id);

    const isMatch = await comparePassword(oldPassword, user.password);
    if (!isMatch) {
      return res.status(401).json({ success: false, error: 'Invalid old password' });
    }

    const hashedPassword = await hashPassword(newPassword);
    db.prepare('UPDATE users SET password = ? WHERE id = ?').run(hashedPassword, req.user.id);
    logAudit(req.user.id, 'CHANGE_PASSWORD', 'users', req.user.id, req.ip);
  }
});

```

```

    res.json({ success: true, message: 'Password updated successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

```

```

router.get('/csrf-token', (req, res) => {
  const sessionId = req.headers['x-session-id'] || uuidv4();
  const token = crypto.randomBytes(32).toString('hex');
  csrfTokens.set(sessionId, token);

```

```

  res.json({
    success: true,
    data: {
      csrfToken: token,
      sessionId: sessionId
    }
  });
});

```

```

module.exports = router;

```

Kode 9. Manajemen Data Siswa (backend/src/routes/student.routes.js)

```

const express = require('express');
const router = express.Router();
const { v4: uuidv4 } = require('uuid');
const db = require('../../config/database');
const { authenticate, authorize } = require('../../middleware/auth');
const cryptoService = require('../../services/crypto.service');

const logAudit = (userId, action, targetTable, targetId, ip, oldValue = null, newValue = null) => {
  db.prepare(`
    INSERT INTO audit_logs (id, user_id, action, target_table, target_id, old_value, new_value,
    ip_address)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
  `).run(uuidv4(), userId, action, targetTable, targetId, oldValue ? JSON.stringify(oldValue) : null,
  newValue ? JSON.stringify(newValue) : null, ip);
};

// GET /api/students
router.get('/', authenticate, authorize('TU', 'ADMIN', 'KEPALA_SEKOLAH', 'GURU'), (req, res) => {
  try {
    const { class: studentClass, major, search } = req.query;
    let query = "SELECT id, nis, nisn, full_name, class, major, enrollment_year, status FROM students
    WHERE status = 'ACTIVE'";
    const params = [];

    if (studentClass) {
      query += " AND class = ?";
      params.push(studentClass);
    }
    if (major) {
      query += " AND major = ?";
      params.push(major);
    }
    if (search) {
      query += " AND (full_name LIKE ? OR nis LIKE ?)";
      params.push(`%${search}%`, `%${search}%`);
    }

    const students = db.prepare(query).all(...params);

```

```

        res.json({ success: true, data: students });
    } catch (err) {
        res.status(500).json({ success: false, error: err.message });
    }
});

// GET /api/students/:id
router.get('/:id', authenticate, (req, res) => {
    try {
        const studentId = req.params.id;
        const student = db.prepare('SELECT * FROM students WHERE id = ?').get(studentId);

        if (!student) {
            return res.status(404).json({ success: false, error: 'Student not found' });
        }

        // Access check: Only authorized roles or the student themselves
        if (!['TU', 'ADMIN', 'KEPALA_SEKOLAH', 'GURU'].includes(req.user.role) && req.user.id !==
student.user_id) {
            return res.status(403).json({ success: false, error: 'Access denied' });
        }

        // Decrypt sensitive fields
        student.nik = cryptoService.decryptField(student.encrypted_nik);
        student.address = cryptoService.decryptField(student.encrypted_address);
        student.phone = cryptoService.decryptField(student.encrypted_phone);

        delete student.encrypted_nik;
        delete student.encrypted_address;
        delete student.encrypted_phone;

        res.json({ success: true, data: student });
    } catch (err) {
        res.status(500).json({ success: false, error: err.message });
    }
});

// POST /api/students
router.post('/', authenticate, authorize('TU', 'ADMIN'), (req, res) => {
    try {
        const { nis, nisan, nama, full_name, tempatLahir, birth_place, tanggalLahir, birth_date, jenisKelamin,
gender, kelas, studentClass, jurusan, major, tahunMasuk, enrollment_year, nik, alamat, address, noHp,
phone, user_id } = req.body;
        const id = uuidv4();

        const finalFullName = nama || full_name || '';
        const finalBirthPlace = tempatLahir || birth_place || '';
        const finalBirthDate = tanggalLahir || birth_date || '';
        const finalGender = jenisKelamin || gender || 'L';
        const finalClass = kelas || studentClass || '';
        const finalMajor = jurusan || major || '';
        const finalYear = tahunMasuk || enrollment_year || new Date().getFullYear();
        const finalNik = nik || '';
        const finalAddress = alamat || address || '';
        const finalPhone = noHp || phone || '';

        const encryptedNik = cryptoService.encryptField(finalNik);
        const encryptedAddress = cryptoService.encryptField(finalAddress);
        const encryptedPhone = cryptoService.encryptField(finalPhone);

        db.prepare(`
            INSERT INTO students (id, nis, nisan, full_name, birth_place, birth_date, gender, class, major,
enrollment_year, encrypted_nik, encrypted_address, encrypted_phone, user_id)

```

```

        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `).run(id, nis, nisan, finalFullName, finalBirthPlace, finalBirthDate, finalGender, finalClass,
    finalMajor, finalYear, encryptedNik, encryptedAddress, encryptedPhone, user_id || null);

    logAudit(req.user.id, 'CREATE_STUDENT', 'students', id, req.ip, null, { nis, full_name: finalFullName
    });

    res.json({ success: true, data: { id, nis, full_name: finalFullName } });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// PUT /api/students/:id
router.put('/:id', authenticate, authorize('TU', 'ADMIN'), (req, res) => {
  try {
    const id = req.params.id;
    const { nama, full_name, kelas, studentClass, jurusan, major } = req.body;

    const finalFullName = nama || full_name || '';
    const finalClass = kelas || studentClass || '';
    const finalMajor = jurusan || major || '';

    const oldData = db.prepare('SELECT full_name, class, major FROM students WHERE id = ?').get(id);

    db.prepare(`
      UPDATE students SET full_name = ?, class = ?, major = ?, updated_at = CURRENT_TIMESTAMP
      WHERE id = ?
    `).run(finalFullName, finalClass, finalMajor, id);

    logAudit(req.user.id, 'UPDATE_STUDENT', 'students', id, req.ip, oldData, { full_name: finalFullName,
    class: finalClass, major: finalMajor });

    res.json({ success: true, message: 'Student updated successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// DELETE /api/students/:id
router.delete('/:id', authenticate, authorize('ADMIN'), (req, res) => {
  try {
    const id = req.params.id;
    db.prepare('UPDATE students SET status = 'INACTIVE', updated_at = CURRENT_TIMESTAMP WHERE id =
    ?').run(id);
    logAudit(req.user.id, 'DELETE_STUDENT', 'students', id, req.ip);
    res.json({ success: true, message: 'Student deactivated' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

module.exports = router;

```

Kode 10. Sertifikat & Ijazah (backend/src/routes/certificate.routes.js)

```

const express = require('express');
const router = express.Router();
const { v4: uuidv4 } = require('uuid');
const multer = require('multer');
const path = require('path');
const fs = require('fs');

```

```

const db = require('../config/database');
const { authenticate, authorize } = require('../middleware/auth');
const cryptoService = require('../services/crypto.service');
const blockchainService = require('../services/blockchain.service');

const uploadDir = path.join(__dirname, '../..../uploads/');
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });
const upload = multer({ dest: uploadDir });

const logAudit = (userId, action, targetTable, targetId, ip) => {
  db.prepare(`
    INSERT INTO audit_logs (id, user_id, action, target_table, target_id, ip_address)
    VALUES (?, ?, ?, ?, ?, ?)
  `).run(uuidv4(), userId, action, targetTable, targetId, ip);
};

// GET /api/certificates
router.get('/', authenticate, authorize('TU', 'ADMIN', 'KEPALA_SEKOLAH'), (req, res) => {
  try {
    const certs = db.prepare(`
      SELECT c.*, s.full_name as student_name, s.nis
      FROM certificates c
      JOIN students s ON c.student_id = s.id
    `).all();
    res.json({ success: true, data: certs });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// POST /api/certificates/issue
router.post('/issue', authenticate, authorize('TU', 'ADMIN', 'KEPALA_SEKOLAH'), upload.single('pdf'),
async (req, res) => {
  try {
    const { student_id, type } = req.body;

    if (!req.file) {
      return res.status(400).json({ success: false, error: 'PDF file is required' });
    }

    const pdfHash = await cryptoService.hashFile(req.file.path);
    const id = uuidv4();
    const certNum = cryptoService.generateCertificateNumber(type, new Date().getFullYear(),
Math.floor(Math.random() * 1000));

    db.prepare(`
      INSERT INTO certificates (id, student_id, type, certificate_number, pdf_hash, issued_by, issued_at)
      VALUES (?, ?, ?, ?, ?, ?, CURRENT_TIMESTAMP)
    `).run(id, student_id, type, certNum, pdfHash, req.user.id);

    logAudit(req.user.id, 'ISSUE_CERTIFICATE', 'certificates', id, req.ip);

    // Cleanup temp file
    fs.unlinkSync(req.file.path);

    res.json({ success: true, data: { id, certificate_number: certNum, pdf_hash: pdfHash } });
  } catch (err) {
    if (req.file) fs.unlinkSync(req.file.path);
    res.status(500).json({ success: false, error: err.message });
  }
});

// POST /api/certificates/:id/approve

```

```

router.post('/:id/approve', authenticate, authorize('KEPALA_SEKOLAH'), async (req, res) => {
  try {
    const id = req.params.id;
    const cert = db.prepare('SELECT * FROM certificates WHERE id = ?').get(id);

    if (!cert) return res.status(404).json({ success: false, error: 'Certificate not found' });
    if (cert.status !== 'PENDING') return res.status(400).json({ success: false, error: 'Certificate is not pending' });

    const metadata = { certNum: cert.certificate_number, type: cert.type };
    const txHash = await blockchainService.issueCertificate(cert.student_id, cert.type, cert.pdf_hash, metadata);

    if (txHash) {
      db.prepare("UPDATE certificates SET status = 'ISSUED', blockchain_tx_hash = ? WHERE id = ?")
        .run(txHash, id);
      logAudit(req.user.id, 'APPROVE_CERTIFICATE', 'certificates', id, req.ip);
      res.json({ success: true, message: 'Certificate approved and minted on blockchain', txHash });
    } else {
      res.status(500).json({ success: false, error: 'Failed to mint on blockchain' });
    }
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// POST /api/certificates/:id/revoke
router.post('/:id/revoke', authenticate, authorize('ADMIN', 'KEPALA_SEKOLAH'), async (req, res) => {
  try {
    const id = req.params.id;
    const { reason } = req.body;

    const cert = db.prepare('SELECT * FROM certificates WHERE id = ?').get(id);
    if (!cert) return res.status(404).json({ success: false, error: 'Certificate not found' });

    db.prepare("UPDATE certificates SET status = 'REVOKED', revoked_at = CURRENT_TIMESTAMP, revoked_reason = ? WHERE id = ?")
      .run(reason, id);

    if (cert.blockchain_tx_hash) {
      await blockchainService.revokeCertificate(cert.pdf_hash, reason);
    }

    logAudit(req.user.id, 'REVOKE_CERTIFICATE', 'certificates', id, req.ip);
    res.json({ success: true, message: 'Certificate revoked successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// POST /api/certificates/verify (PUBLIC)
router.post('/verify', async (req, res) => {
  try {
    const { hash } = req.body;
    if (!hash) {
      return res.status(400).json({ success: false, error: 'Hash is required' });
    }

    // Check DB
    const dbCert = db.prepare('SELECT c.*, s.full_name as student_name FROM certificates c JOIN students s ON c.student_id = s.id WHERE c.pdf_hash = ?').get(hash);

    // Check Blockchain

```

```

const bcCert = await blockchainService.verifyCertificate(hash);

// If neither exists, 404
if (!dbCert && (!bcCert || !bcCert.isValid)) {
  return res.status(404).json({ success: false, error: 'Not found' });
}

// If revoked, 410
if (dbCert && dbCert.status === 'REVOKED') {
  return res.status(410).json({ success: false, message: 'Sertifikat telah dicabut.', reason:
dbCert.revoked_reason });
}

res.json({
  success: true,
  studentName: dbCert ? dbCert.student_name : (bcCert ? bcCert.studentId : 'Unknown'),
  type: dbCert ? dbCert.type : (bcCert ? bcCert.certType : 'Unknown'),
  issuer: 'SMK Negeri (Blockchain Verified)',
  txHash: dbCert ? dbCert.blockchain_tx_hash : null
});
} catch (err) {
  res.status(500).json({ success: false, error: err.message });
}
});

// GET /api/certificates/:id
router.get('/:id', authenticate, (req, res) => {
  try {
    const cert = db.prepare('SELECT * FROM certificates WHERE id = ?').get(req.params.id);
    if (!cert) return res.status(404).json({ success: false, error: 'Not found' });
    res.json({ success: true, data: cert });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

module.exports = router;

```

Kode 11. Audit Log & WAF Logs (backend/src/routes/audit.routes.js)

```

const express = require('express');
const router = express.Router();
const db = require('../../config/database');
const { authenticate, authorize } = require('../../middleware/auth');

// GET /api/audit/logs
router.get('/logs', authenticate, authorize('ADMIN', 'KEPALA_SEKOLAH'), (req, res) => {
  try {
    const limit = parseInt(req.query.limit) || 50;
    const offset = parseInt(req.query.offset) || 0;

    const logs = db.prepare(`
      SELECT a.*, u.username, u.full_name
      FROM audit_logs a
      LEFT JOIN users u ON a.user_id = u.id
      ORDER BY a.created_at DESC
      LIMIT ? OFFSET ?
    `).all(limit, offset);

    res.json({ success: true, data: logs });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

```



```

    }
  });

// GET /api/audit/waf
router.get('/waf', authenticate, authorize('ADMIN'), (req, res) => {
  try {
    const limit = parseInt(req.query.limit) || 50;
    const offset = parseInt(req.query.offset) || 0;

    const logs = db.prepare(`
      SELECT * FROM waf_logs
      ORDER BY created_at DESC
      LIMIT ? OFFSET ?
    `).all(limit, offset);

    res.json({ success: true, data: logs });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// GET /api/audit/waf/stats
router.get('/waf/stats', authenticate, authorize('ADMIN'), (req, res) => {
  try {
    const totalBlocked = db.prepare("SELECT count(*) as count FROM waf_logs WHERE blocked = 1").get().count;

    const byAttackType = db.prepare("SELECT attack_type, count(*) as count FROM waf_logs GROUP BY attack_type").all();

    const bySeverity = db.prepare("SELECT severity, count(*) as count FROM waf_logs GROUP BY severity").all();

    res.json({
      success: true,
      data: {
        totalBlocked,
        byAttackType,
        bySeverity
      }
    });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// GET /api/audit/activity
router.get('/activity', authenticate, authorize('ADMIN'), (req, res) => {
  try {
    const summary = db.prepare("SELECT action, count(*) as count FROM audit_logs GROUP BY action").all();
    res.json({ success: true, data: summary });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

module.exports = router;

```

Kode 12. Manajemen Nilai (backend/src/routes/grade.routes.js)

```

const express = require('express');
const router = express.Router();

```

```

const { v4: uuidv4 } = require('uuid');
const db = require('../config/database');
const { authenticate, authorize } = require('../middleware/auth');
const blockchainService = require('../services/blockchain.service');

const logAudit = (userId, action, targetTable, targetId, ip, oldValue = null, newValue = null) => {
  db.prepare(`
    INSERT INTO audit_logs (id, user_id, action, target_table, target_id, old_value, new_value,
    ip_address)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
  `).run(uuidv4(), userId, action, targetTable, targetId, oldValue ? JSON.stringify(oldValue) : null,
  newValue ? JSON.stringify(newValue) : null, ip);
};

// GET /api/grades
router.get('/', authenticate, authorize('GURU', 'TU', 'ADMIN', 'KEPALA_SEKOLAH'), (req, res) => {
  try {
    const { studentId, subjectId, semester } = req.query;
    let query = "SELECT * FROM grades WHERE 1=1";
    const params = [];

    if (studentId) { query += " AND student_id = ?"; params.push(studentId); }
    if (subjectId) { query += " AND subject_id = ?"; params.push(subjectId); }
    if (semester) { query += " AND semester = ?"; params.push(semester); }

    const grades = db.prepare(query).all(...params);
    res.json({ success: true, data: grades });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// GET /api/grades/student/:studentId
router.get('/student/:studentId', authenticate, (req, res) => {
  try {
    const { studentId } = req.params;

    // Students can only see their own grades
    if (req.user.role === 'SISWA') {
      const student = db.prepare('SELECT user_id FROM students WHERE id = ?').get(studentId);
      if (!student || student.user_id !== req.user.id) {
        return res.status(403).json({ success: false, error: 'Access denied' });
      }
    }

    const grades = db.prepare(`
      SELECT g.*, s.name as subject_name, s.code as subject_code
      FROM grades g
      JOIN subjects s ON g.subject_id = s.id
      WHERE g.student_id = ?
    `).all(studentId);

    res.json({ success: true, data: grades });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// POST /api/grades
router.post('/', authenticate, authorize('GURU'), (req, res) => {
  try {
    const { student_id, subject_id, semester, score, grade_letter } = req.body;
    const id = uuidv4();

```

```

    db.prepare(`
      INSERT INTO grades (id, student_id, subject_id, semester, score, grade_letter, teacher_id)
      VALUES (?, ?, ?, ?, ?, ?, ?)
    `).run(id, student_id, subject_id, semester, score, grade_letter, req.user.id);

    logAudit(req.user.id, 'CREATE_GRADE', 'grades', id, req.ip, null, { score, grade_letter });

    res.json({ success: true, data: { id } });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// PUT /api/grades/:id
router.put('/:id', authenticate, authorize('GURU'), async (req, res) => {
  try {
    const id = req.params.id;
    const { score, grade_letter } = req.body;

    const oldGrade = db.prepare('SELECT score, grade_letter, teacher_id, student_id, subject_id FROM
    grades WHERE id = ?').get(id);
    if (!oldGrade) return res.status(404).json({ success: false, error: 'Grade not found' });

    if (oldGrade.teacher_id !== req.user.id && req.user.role !== 'ADMIN') {
      return res.status(403).json({ success: false, error: 'Can only update own grades' });
    }

    db.prepare('UPDATE grades SET score = ?, grade_letter = ?, updated_at = CURRENT_TIMESTAMP WHERE id =
    ?')
      .run(score, grade_letter, id);

    logAudit(req.user.id, 'UPDATE_GRADE', 'grades', id, req.ip,
      { score: oldGrade.score, grade_letter: oldGrade.grade_letter },
      { score, grade_letter }
    );

    // Optionally log to blockchain
    const txHash = await blockchainService.logGradeChange(oldGrade.student_id, oldGrade.subject_id,
    oldGrade.score, score, req.user.id);
    if (txHash) {
      db.prepare('UPDATE grades SET blockchain_tx_hash = ? WHERE id = ?').run(txHash, id);
    }

    res.json({ success: true, message: 'Grade updated successfully' });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});

// GET /api/grades/report/:studentId
router.get('/report/:studentId', authenticate, (req, res) => {
  try {
    const student = db.prepare('SELECT * FROM students WHERE id = ?').get(req.params.studentId);
    if (!student) return res.status(404).json({ success: false, error: 'Student not found' });

    const grades = db.prepare(`
      SELECT g.*, s.name as subject_name
      FROM grades g
      JOIN subjects s ON g.subject_id = s.id
      WHERE g.student_id = ?
    `).all(req.params.studentId);

```

```
const report = {
  student_name: student.full_name,
  nis: student.nis,
  class: student.class,
  grades: grades
};

res.json({ success: true, data: report });
} catch (err) {
  res.status(500).json({ success: false, error: err.message });
}
});

module.exports = router;
```

BACKEND - SERVICES

Kode 13. Enkripsi AES-256 & Hashing SHA-256 (backend/src/services/crypto.service.js)

```
const fs = require('fs');
const crypto = require('crypto');
const { encrypt, decrypt, hashDocument } = require('../middleware/encryption');

class CryptoService {
  hashFile(filePath) {
    return new Promise((resolve, reject) => {
      const hash = crypto.createHash('sha256');
      const stream = fs.createReadStream(filePath);

      stream.on('data', (data) => {
        hash.update(data);
      });

      stream.on('end', () => {
        resolve(hash.digest('hex'));
      });

      stream.on('error', (err) => {
        reject(err);
      });
    });
  }

  hashBuffer(buffer) {
    return hashDocument(buffer);
  }

  encryptField(plaintext) {
    return encrypt(plaintext);
  }

  decryptField(ciphertext) {
    return decrypt(ciphertext);
  }

  generateCertificateNumber(type, year, sequence) {
    let prefix = 'IJZ';
    if (type === 'TRANSKRIP') prefix = 'TRK';
    else if (type === 'SERTIFIKAT_PKL') prefix = 'PKL';
    else if (type === 'SERTIFIKAT_MAGANG') prefix = 'MAG';

    const paddedSeq = String(sequence).padStart(5, '0');
    return `${prefix}-${year}-${paddedSeq}`;
  }
}

module.exports = new CryptoService();
```

Kode 14. Integrasi Ethereum Blockchain (backend/src/services/blockchain.service.js)

```
const { ethers } = require('ethers');

// Mock ABI in case file is not found
let CertificateRegistryABI = [
  "function issueCertificate(string studentId, string certType, bytes32 documentHash, string metadata)",
  "function verifyCertificate(bytes32 documentHash) view returns (bool isValid, string studentId, string certType, uint256 timestamp, address issuer, bool isRevoked)",
];
```

```

    "function getCertificatesByStudent(string studentId) view returns (bytes32[])",
    "function revokeCertificate(bytes32 documentHash, string reason)"
  ];

  try {
    const artifact =
    require('../../../../blockchain/artifacts/contracts/CertificateRegistry.sol/CertificateRegistry.json');
    if (artifact && artifact.abi) {
      CertificateRegistryABI = artifact.abi;
    }
  } catch (error) {
    console.warn('CertificateRegistry ABI not found at expected path. Using default ABI.');
```

```

  }
}

class BlockchainService {
```

```

  constructor() {
    this.provider = null;
    this.wallet = null;
    this.contract = null;
    this.initialized = false;
  }
```

```

  init() {
```

```

    try {
      const rpcUrl = process.env.BLOCKCHAIN_RPC_URL || 'http://127.0.0.1:8545';
      const privateKey = process.env.DEPLOYER_PRIVATE_KEY;
      const contractAddress = process.env.CONTRACT_ADDRESS;
```

```

      if (!privateKey || !contractAddress) {
        console.warn('Blockchain credentials not fully provided in .env');
        return;
      }

```

```

      this.provider = new ethers.JsonRpcProvider(rpcUrl);
      this.wallet = new ethers.Wallet(privateKey, this.provider);
      this.contract = new ethers.Contract(contractAddress, CertificateRegistryABI, this.wallet);
      this.initialized = true;
      console.log('Blockchain service initialized');
    } catch (err) {
      console.error('Failed to initialize blockchain service:', err);
    }
  }
}
```

```

  getContract() {
    if (!this.initialized) this.init();
    return this.contract;
  }
}
```

```

  async issueCertificate(studentId, certType, pdfHash, metadata) {
```

```

    try {
      const contract = this.getContract();
      if (!contract) return null;
```

```

      const hashBytes32 = pdfHash.startsWith('0x') ? pdfHash : '0x' + pdfHash;
      const tx = await contract.issueCertificate(studentId, certType, hashBytes32,
```

```

JSON.stringify(metadata));
```

```

      const receipt = await tx.wait();
      return receipt.hash;
```

```

    } catch (error) {
      console.error('Blockchain issueCertificate error:', error);
      return null;
    }
  }
}
```

```

async verifyCertificate(pdfHash) {
  try {
    const contract = this.getContract();
    if (!contract) return null;

    const hashBytes32 = pdfHash.startsWith('0x') ? pdfHash : '0x' + pdfHash;
    const result = await contract.verifyCertificate(hashBytes32);
    return {
      isValid: result.isValid,
      studentId: result.studentId,
      certType: result.certType,
      timestamp: Number(result.timestamp),
      issuer: result.issuer
    };
  } catch (error) {
    console.error('Blockchain verifyCertificate error:', error);
    return null;
  }
}

async getCertificateDetails(pdfHash) {
  try {
    const contract = this.getContract();
    if (!contract) return null;

    const hashBytes32 = pdfHash.startsWith('0x') ? pdfHash : '0x' + pdfHash;
    const result = await contract.getCertificateDetails(hashBytes32);
    return {
      studentId: result.studentId,
      certType: result.certType,
      metadata: result.metadata,
      timestamp: Number(result.timestamp),
      issuer: result.issuer,
      isRevoked: result.isRevoked
    };
  } catch (error) {
    console.error('Blockchain getCertificateDetails error:', error);
    return null;
  }
}

async revokeCertificate(pdfHash, reason) {
  try {
    const contract = this.getContract();
    if (!contract) return null;

    const hashBytes32 = pdfHash.startsWith('0x') ? pdfHash : '0x' + pdfHash;
    const tx = await contract.revokeCertificate(hashBytes32, reason);
    const receipt = await tx.wait();
    return receipt.hash;
  } catch (error) {
    console.error('Blockchain revokeCertificate error:', error);
    return null;
  }
}

async logGradeChange(studentId, subjectId, oldScore, newScore, teacherId) {
  // Implement if GradeAuditLog contract is available
  // Mock implementation for now
  console.log(`Mock Blockchain Grade Log: \${studentId} \${subjectId} \${oldScore}->\${newScore} by \${teacherId}`);
  return "0xmocktxhash1234567890abcdef";
}

```

```
}  
}
```

```
module.exports = new BlockchainService();
```

BACKEND - DATABASE SEEDER

Kode 15. Data Awal (Users, Students, Subjects) (backend/src/seed.js)

```
require('dotenv').config();
const { v4: uuidv4 } = require('uuid');
const db = require('./config/database');
const { hashPassword } = require('./middleware/encryption');
const cryptoService = require('./services/crypto.service');

async function seed() {
  // ✅ Inisialisasi database terlebih dahulu
  await db.initialize();

  console.log('Starting database seed...');

  // Clear existing (optional, but good for testing)
  db.prepare('DELETE FROM certificates').run();
  db.prepare('DELETE FROM grades').run();
  db.prepare('DELETE FROM students').run();
  db.prepare('DELETE FROM subjects').run();
  db.prepare('DELETE FROM audit_logs').run();
  db.prepare('DELETE FROM waf_logs').run();
  db.prepare('DELETE FROM users').run();

  const users = [
    { username: 'admin', password: 'Admin@123', role: 'ADMIN', full_name: 'Administrator Sistem' },
    { username: 'kepsek', password: 'Kepsek@123', role: 'KEPALA_SEKOLAH', full_name: 'Dr. Ahmad Suryadi, M.Pd' },
    { username: 'guru1', password: 'Guru@123', role: 'GURU', full_name: 'Siti Nurhaliza, S.Pd' },
    { username: 'tu1', password: 'TataUsaha@123', role: 'TU', full_name: 'Budi Santoso' },
    { username: 'siswa1', password: 'Siswa@123', role: 'SISWA', full_name: 'Alamsyah Sahlan' },
    { username: 'dudi1', password: 'Dudi@123', role: 'DUDI', full_name: 'PT Teknologi Nusantara' }
  ];

  const userIds = {};

  for (const u of users) {
    const id = uuidv4();
    userIds[u.username] = id;
    const hashed = await hashPassword(u.password);

    db.prepare(`
      INSERT INTO users (id, username, password, full_name, role)
      VALUES (?, ?, ?, ?, ?)
    `).run(id, u.username, hashed, u.full_name, u.role);
    console.log(`Created user: ${u.username} (${u.role})`);
  }

  const subjects = [
    { code: 'MK-01', name: 'Matematika', major: 'UMUM', semester: 1 },
    { code: 'MK-02', name: 'Bahasa Indonesia', major: 'UMUM', semester: 1 },
    { code: 'MK-03', name: 'Bahasa Inggris', major: 'UMUM', semester: 1 },
    { code: 'RPL-01', name: 'Pemrograman Web', major: 'RPL', semester: 1 },
    { code: 'RPL-02', name: 'Basis Data', major: 'RPL', semester: 1 },
    { code: 'TKJ-01', name: 'Jaringan Komputer', major: 'TKJ', semester: 1 }
  ];

  const subjectIds = {};

  for (const s of subjects) {
    const id = uuidv4();
    subjectIds[s.code] = id;
```

```

    db.prepare(`
        INSERT INTO subjects (id, code, name, major, semester)
        VALUES (?, ?, ?, ?, ?)
    `).run(id, s.code, s.name, s.major, s.semester);
}
console.log('Created subjects');

const students = [
    { nis: '1001', nisan: '001001', full_name: 'Alamsyah Sahlan', class: 'XII-RPL', major: 'RPL', nik:
'3201010101010001', address: 'Jl. Merdeka No 1', phone: '081234567890', user_id: userIds['siswa1'] },
    { nis: '1002', nisan: '001002', full_name: 'Budi Darmawan', class: 'XII-RPL', major: 'RPL', nik:
'3201010101010002', address: 'Jl. Merdeka No 2', phone: '081234567891', user_id: null },
    { nis: '1003', nisan: '001003', full_name: 'Citra Kirana', class: 'XII-TKJ', major: 'TKJ', nik:
'3201010101010003', address: 'Jl. Merdeka No 3', phone: '081234567892', user_id: null },
    { nis: '1004', nisan: '001004', full_name: 'Dewi Lestari', class: 'XII-TKJ', major: 'TKJ', nik:
'3201010101010004', address: 'Jl. Merdeka No 4', phone: '081234567893', user_id: null },
    { nis: '1005', nisan: '001005', full_name: 'Eko Patrio', class: 'XII-RPL', major: 'RPL', nik:
'3201010101010005', address: 'Jl. Merdeka No 5', phone: '081234567894', user_id: null }
];

const studentIds = {};

for (const st of students) {
    const id = uuidv4();
    studentIds[st.nis] = id;

    db.prepare(`
        INSERT INTO students (id, nis, nisan, full_name, class, major, enrollment_year, encrypted_nik,
encrypted_address, encrypted_phone, user_id)
        VALUES (?, ?, ?, ?, ?, ?, 2021, ?, ?, ?, ?)
    `).run(
        id, st.nis, st.nisan, st.full_name, st.class, st.major,
        cryptoService.encryptField(st.nik),
        cryptoService.encryptField(st.address),
        cryptoService.encryptField(st.phone),
        st.user_id
    );
}
console.log('Created 5 students with encrypted PII');

// Grades for siswa1
const grades = [
    { student_id: studentIds['1001'], subject_id: subjectIds['MK-01'], score: 85.5, grade_letter: 'A' },
    { student_id: studentIds['1001'], subject_id: subjectIds['RPL-01'], score: 90.0, grade_letter: 'A' }
];

for (const g of grades) {
    const id = uuidv4();
    db.prepare(`
        INSERT INTO grades (id, student_id, subject_id, semester, score, grade_letter, teacher_id)
        VALUES (?, ?, ?, 1, ?, ?, ?)
    `).run(id, g.student_id, g.subject_id, g.score, g.grade_letter, userIds['guru1']);
}
console.log('Created sample grades');

console.log('\nSeed completed successfully!');
console.log('-----');
console.log('Default Admin -> admin : Admin@123');
console.log('Default Kepsek -> kepsek : Kepsek@123');
console.log('Default Guru -> guru1 : Guru@123');
console.log('Default TU -> tu1 : TataUsaha@123');
console.log('Default Siswa -> siswa1 : Siswa@123');
console.log('Default DUDI -> dudi1 : Dudi@123');

```

```
    console.log('-----');  
  }  
  
  seed().catch(err => {  
    console.error('Seed failed:', err);  
    process.exit(1);  
  });
```

FRONTEND - KONFIGURASI

Kode 16. Vite Configuration (frontend/vite.config.js)

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:3001',
        changeOrigin: true
      }
    }
  }
});
```

Kode 17. Entry Point React (frontend/src/main.jsx)

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
import { BrowserRouter } from 'react-router-dom'
import { AuthProvider } from './contexts/AuthContext.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>,
)
```

Kode 18. App Router (frontend/src/App.jsx)

```
import { Routes, Route, Navigate } from 'react-router-dom'
import Login from './pages/Login'
import Dashboard from './pages/Dashboard'
import Students from './pages/Students'
import Grades from './pages/Grades'
import Certificates from './pages/Certificates'
import VerifyDocument from './pages/VerifyDocument'
import AuditLog from './pages/AuditLog'
import WafDashboard from './pages/WafDashboard'
import MfaSetup from './pages/MfaSetup'
import ProtectedRoute from './components/ProtectedRoute'
import Sidebar from './components/Sidebar'
import { useAuth } from './contexts/AuthContext'

function App() {
  const { isAuthenticated } = useAuth()

  return (
    <div className="flex h-screen bg-slate-50">
```

```

{isAuthenticated && <Sidebar />}
<div className={`flex-1 overflow-y-auto ${isAuthenticated ? 'w-full' : 'w-full'}}`>
  <Routes>
    <Route path="/login" element={<Login />} />
    <Route path="/verify" element={<VerifyDocument />} />

    <Route path="/" element={<Navigate to="/dashboard" replace />} />

    <Route path="/dashboard" element={
      <ProtectedRoute>
        <Dashboard />
      </ProtectedRoute>
    } />

    <Route path="/students" element={
      <ProtectedRoute roles={['TU', 'ADMIN', 'KEPALA_SEKOLAH', 'GURU']}>
        <Students />
      </ProtectedRoute>
    } />

    <Route path="/grades" element={
      <ProtectedRoute roles={['GURU', 'ADMIN']}>
        <Grades />
      </ProtectedRoute>
    } />

    <Route path="/certificates" element={
      <ProtectedRoute roles={['TU', 'ADMIN', 'KEPALA_SEKOLAH']}>
        <Certificates />
      </ProtectedRoute>
    } />

    <Route path="/audit" element={
      <ProtectedRoute roles={['ADMIN', 'KEPALA_SEKOLAH']}>
        <AuditLog />
      </ProtectedRoute>
    } />

    <Route path="/waf" element={
      <ProtectedRoute roles={['ADMIN']}>
        <WafDashboard />
      </ProtectedRoute>
    } />

    <Route path="/mfa-setup" element={
      <ProtectedRoute>
        <MfaSetup />
      </ProtectedRoute>
    } />
  </Routes>
</div>
</div>
)
}

export default App

```

Kode 19. Axios API Instance (frontend/src/utlis/api.js)

```

import axios from 'axios';

const api = axios.create({

```

```

    baseURL: '/api'
  });

  api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    config.headers['x-csrf-token'] = 'dummy-token';
    return config;
  });

  api.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response && error.response.status === 401) {
        localStorage.removeItem('token');
        window.location.href = '/login';
      }
      return Promise.reject(error);
    }
  );
};

export default api;

```

Kode 20. Authentication Context (frontend/src/contexts/AuthContext.jsx)

```

import React, { createContext, useContext, useState, useEffect } from 'react';
import api from '../utils/api';

const AuthContext = createContext();

export const useAuth = () => useContext(AuthContext);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token') || null);

  useEffect(() => {
    if (token) {
      try {
        const payload = JSON.parse(atob(token.split('.')[1]));
        setUser(payload);
      } catch (e) {
        setToken(null);
        localStorage.removeItem('token');
      }
    } else {
      setUser(null);
    }
  }, [token]);

  const login = async (username, password, mfaToken) => {
    try {
      const response = await api.post('/auth/login', { username, password, mfaToken });

      if (response.data.mfa_required) {
        return { mfaRequired: true };
      }

      const newToken = response.data.data.token;
      setToken(newToken);
    }
  };

```

```
        localStorage.setItem('token', newToken);
        return { success: true };
    } catch (error) {
        console.error("Login error", error);
        throw error;
    }
};

const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
};

const hasRole = (role) => {
    if (Array.isArray(role)) {
        return role.includes(user?.role);
    }
    return user?.role === role;
};

return (
    <AuthContext.Provider value={{ user, token, login, logout, isAuthenticated: !!token, hasRole }}>
        {children}
    </AuthContext.Provider>
);
};
```

FRONTEND - KOMPONEN

Kode 21. Sidebar Navigation (frontend/src/components/Sidebar.jsx)

```
import React from 'react';
import { NavLink } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import { LayoutDashboard, Users, GraduationCap, Award, ClipboardList, Shield, Logout, Settings } from
' lucide-react';

const Sidebar = () => {
  const { user, logout, hasRole } = useAuth();

  const navItems = [
    { to: '/dashboard', label: 'Dashboard', icon: <LayoutDashboard size={20} />, roles: ['ADMIN',
'KEPALA_SEKOLAH', 'GURU', 'TU', 'SISWA'] },
    { to: '/students', label: 'Data Siswa', icon: <Users size={20} />, roles: ['ADMIN', 'KEPALA_SEKOLAH',
'GURU', 'TU'] },
    { to: '/grades', label: 'Manajemen Nilai', icon: <GraduationCap size={20} />, roles: ['ADMIN', 'GURU']
},
    { to: '/certificates', label: 'Sertifikat & Ijazah', icon: <Award size={20} />, roles: ['ADMIN',
'KEPALA_SEKOLAH', 'TU'] },
    { to: '/audit', label: 'Audit Log', icon: <ClipboardList size={20} />, roles: ['ADMIN',
'KEPALA_SEKOLAH'] },
    { to: '/waf', label: 'WAF Dashboard', icon: <Shield size={20} />, roles: ['ADMIN'] },
    { to: '/mfa-setup', label: 'MFA Setup', icon: <Settings size={20} />, roles: ['ADMIN',
'KEPALA_SEKOLAH', 'GURU', 'TU', 'SISWA'] },
  ];

  return (
    <div className="w-64 bg-slate-900 text-white h-screen flex flex-col transition-all duration-300">
      <div className="p-4 flex items-center gap-3 border-b border-slate-700">
        <Shield className="text-blue-400" size={28} />
        <h1 className="font-bold text-lg whitespace-nowrap">SMK SecureAdmin</h1>
      </div>

      <div className="flex-1 overflow-y-auto py-4">
        <nav className="space-y-1 px-2">
          {navItems.filter(item => hasRole(item.roles)).map((item) => (
            <NavLink
              key={item.to}
              to={item.to}
              className={({ isActive }) =>
                `flex items-center gap-3 px-3 py-2.5 rounded-lg transition-colors ${
                  isActive ? 'bg-blue-600 text-white' : 'text-slate-300 hover:bg-slate-800 hover:text-
white'
                }`
            >
              {item.icon}
              <span>{item.label}</span>
            </NavLink>
          ))}
        </nav>
      </div>

      <div className="p-4 border-t border-slate-700">
        <div className="mb-4">
          <p className="text-sm font-medium text-white">{user?.username || 'User'}</p>
          <span className="inline-block px-2 py-1 mt-1 text-xs font-semibold bg-slate-700 text-slate-300
rounded-md">
            {user?.role || 'Role'}
          </span>
        </div>
      </div>
    </div>
  );
};
```



```

        </div>
        <button
          onClick={logout}
          className="flex items-center gap-2 w-full px-3 py-2 text-sm text-red-400 hover:bg-slate-800
hover:text-red-300 rounded-lg transition-colors"
        >
          <Logout size={18} />
          Logout
        </button>
      </div>
    </div>
  );
};

export default Sidebar;

```

Kode 22. Protected Route Component (frontend/src/components/ProtectedRoute.jsx)

```

import React from 'react';
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import { ShieldAlert } from 'lucide-react';

const ProtectedRoute = ({ children, roles }) => {
  const { isAuthenticated, hasRole } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (roles && !hasRole(roles)) {
    return (
      <div className="flex flex-col items-center justify-center h-full bg-slate-50 text-slate-800">
        <ShieldAlert size={64} className="text-red-500 mb-4" />
        <h1 className="text-3xl font-bold mb-2">Access Denied</h1>
        <p className="text-slate-600 mb-6">You do not have permission to view this page.</p>
        <button
          onClick={() => window.history.back()}
          className="px-4 py-2 bg-blue-600 text-white rounded hover:bg-blue-700 transition"
        >
          Go Back
        </button>
      </div>
    );
  }

  return children;
};

export default ProtectedRoute;

```

FRONTEND - HALAMAN

Kode 23. Halaman Login (frontend/src/pages/Login.jsx)

```
import React, { useState } from 'react';
import { useNavigate, useLocation, Link } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import { Shield, Lock, User, AlertCircle, FileText } from 'lucide-react';

const Login = () => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [mfaToken, setMfaToken] = useState('');
  const [mfaRequired, setMfaRequired] = useState(false);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const from = location.state?.from?.pathname || '/dashboard';

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      const result = await login(username, password, mfaToken);
      if (result.mfaRequired) {
        setMfaRequired(true);
      } else {
        navigate(from, { replace: true });
      }
    } catch (err) {
      setError(err.response?.data?.error || 'Login failed. Please check your credentials.');
```

```
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-slate-900 bg-gradient-to-br from-slate-900 to-slate-800 flex items-center justify-center p-4">
      <div className="w-full max-w-md">
        <div className="text-center mb-8">
          <Shield className="w-16 h-16 text-blue-500 mx-auto mb-4" />
          <h1 className="text-3xl font-bold text-white mb-2">Sistem Administrasi SMK</h1>
          <p className="text-slate-400">Cloud Security Architecture</p>
        </div>

        <div className="bg-white rounded-xl shadow-2xl p-8 mb-6">
          <h2 className="text-xl font-bold text-slate-800 mb-6 text-center">Login to your account</h2>

          {error && (
            <div className="bg-red-50 text-red-600 p-4 rounded-lg mb-6 flex items-start gap-3">
              <AlertCircle className="w-5 h-5 flex-shrink-0 mt-0.5" />
              <span className="text-sm">{error}</span>
            </div>
          )}

          <form onSubmit={handleSubmit} className="space-y-5">
            <div>
```

```

<label className="block text-sm font-medium text-slate-700 mb-1">Username</label>
<div className="relative">
  <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
    <User className="h-5 w-5 text-slate-400" />
  </div>
  <input
    type="text"
    required
    disabled={mfaRequired}
    value={username}
    onChange={(e) => setUsername(e.target.value)}
    className="pl-10 w-full p-2.5 border border-slate-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-blue-500 disabled:bg-slate-100"
    placeholder="Enter username"
  />
</div>
</div>

<div>
  <label className="block text-sm font-medium text-slate-700 mb-1">Password</label>
  <div className="relative">
    <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
      <Lock className="h-5 w-5 text-slate-400" />
    </div>
    <input
      type="password"
      required
      disabled={mfaRequired}
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      className="pl-10 w-full p-2.5 border border-slate-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-blue-500 disabled:bg-slate-100"
      placeholder="••••••••"
    />
  </div>
</div>

{mfaRequired && (
  <div>
    <label className="block text-sm font-medium text-slate-700 mb-1">MFA Token (Authenticator
App)</label>
    <input
      type="text"
      required
      value={mfaToken}
      onChange={(e) => setMfaToken(e.target.value)}
      className="w-full p-2.5 border border-slate-300 rounded-lg focus:ring-2 focus:ring-blue-
500 focus:border-blue-500"
      placeholder="Enter 6-digit code"
    />
  </div>
)}

<button
  type="submit"
  disabled={loading}
  className="w-full bg-blue-600 hover:bg-blue-700 text-white font-semibold py-2.5 rounded-lg
transition-colors flex items-center justify-center gap-2"
>
  {loading ? 'Authenticating...' : (mfaRequired ? 'Verify Token' : 'Sign In')}
</button>
</form>

```

```

        <div className="mt-6 pt-6 border-t border-slate-100">
            <Link to="/verify" className="flex items-center justify-center gap-2 text-blue-600 hover:text-blue-700 text-sm font-medium transition-colors">
                <FileText className="w-4 h-4" />
                Go to Document Verification Portal
            </Link>
        </div>
    </div>

    <div className="text-center">
        <div className="inline-flex items-center justify-center gap-2 px-4 py-2 bg-slate-800/50 rounded-full border border-slate-700 mb-6">
            <Shield className="w-4 h-4 text-green-400" />
            <span className="text-xs text-slate-300 font-medium">Protected by WAF & Zero Trust Architecture</span>
        </div>

        <div className="text-xs text-slate-500 text-left bg-slate-800/80 p-4 rounded-lg border border-slate-700">
            <p className="font-semibold text-slate-300 mb-2">Demo Credentials:</p>
            <ul className="space-y-1">
                <li>Admin: admin / Admin@123</li>
                <li>Kepala Sekolah: kepek / Kepsek@123</li>
                <li>Guru: guru1 / Guru@123</li>
                <li>TU: tu1 / TataUsaha@123</li>
                <li>Siswa: siswa1 / Siswa@123</li>
            </ul>
        </div>
    </div>
</div>
</div>
);
};

export default Login;

```

Kode 24. Dashboard Utama (frontend/src/pages/Dashboard.jsx)

```

import React, { useState, useEffect } from 'react';
import { useAuth } from '../contexts/AuthContext';
import api from '../utils/api';
import { Users, Award, Shield, FileEdit, CheckCircle, XCircle } from 'lucide-react';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts';

const Dashboard = () => {
    const { user } = useAuth();
    const [stats, setStats] = useState({
        students: 0,
        certificates: 0,
        wafAttacks: 0,
        gradeChanges: 0,
    });
    const [recentLogs, setRecentLogs] = useState([]);
    const [wafChartData, setWafChartData] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const fetchDashboardData = async () => {
            try {
                const [studentsRes, certsRes, wafRes, logsRes] = await Promise.all([
                    api.get('/students').catch(() => ({ data: [] })),
                    api.get('/certificates').catch(() => ({ data: [] })),

```

```

    api.get('/audit/waf/stats').catch(() => ({ data: { totalBlocked: 0, byType: [] } })),
    api.get('/audit/logs?limit=10').catch(() => ({ data: [] })))
  });

  setStats({
    students: studentsRes.data.length || 0,
    certificates: certsRes.data.length || 0,
    wafAttacks: wafRes.data.totalBlocked || 0,
    gradeChanges: 0 // Mock for now unless API provides it
  });

  if (wafRes.data.byType) {
    setWafChartData(wafRes.data.byType);
  } else {
    setWafChartData([
      { name: 'SQLi', count: 12 },
      { name: 'XSS', count: 8 },
      { name: 'CSRF', count: 3 },
      { name: 'Rate Limit', count: 25 },
    ]);
  }

  setRecentLogs(logsRes.data || []);
} catch (error) {
  console.error("Error fetching dashboard data", error);
} finally {
  setLoading(false);
}
});

fetchDashboardData();
}, []);

const StatCard = ({ title, value, icon: Icon, colorClass }) => (
  <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6 flex items-center gap-4">
    <div className={`p-4 rounded-lg ${colorClass}`}>
      <Icon className="w-8 h-8 text-white" />
    </div>
    <div>
      <p className="text-sm font-medium text-slate-500 mb-1">{title}</p>
      <h3 className="text-2xl font-bold text-slate-800">{value}</h3>
    </div>
  </div>
);

if (loading) {
  return <div className="p-8 text-center text-slate-500">Loading dashboard...</div>;
}

return (
  <div className="p-8">
    <div className="mb-8">
      <h1 className="text-2xl font-bold text-slate-800">Welcome, {user?.username}</h1>
      <p className="text-slate-500">Role: {user?.role}</p>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
      <StatCard title="Total Siswa" value={stats.students} icon={Users} colorClass="bg-blue-500" />
      <StatCard title="Sertifikat Diterbitkan" value={stats.certificates} icon={Award} colorClass="bg-green-500" />
      <StatCard title="Serangan WAF Diblokir" value={stats.wafAttacks} icon={Shield} colorClass="bg-red-500" />
    </div>
  </div>
);

```

```

        <StatCard title="Perubahan Nilai Bulan Ini" value={stats.gradeChanges} icon={FileEdit}
colorClass="bg-yellow-500" />
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        <div className="lg:col-span-2 bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <h3 className="text-lg font-semibold text-slate-800 mb-4">WAF Attack Summary</h3>
            <div className="h-64">
                <ResponsiveContainer width="100%" height="100%">
                    <BarChart data={wafChartData}>
                        <CartesianGrid strokeDasharray="3 3" vertical={false} />
                        <XAxis dataKey="name" axisLine={false} tickLine={false} />
                        <YAxis axisLine={false} tickLine={false} />
                        <Tooltip cursor={{ fill: '#f1f5f9' }} />
                        <Bar dataKey="count" fill="#ef4444" radius={[4, 4, 0, 0]} />
                    </BarChart>
                </ResponsiveContainer>
            </div>
        </div>

        <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <h3 className="text-lg font-semibold text-slate-800 mb-4">Security Status</h3>
            <div className="space-y-4">
                <div className="flex items-center justify-between">
                    <span className="text-slate-600 font-medium">WAF Protection</span>
                    <div className="flex items-center gap-1.5 text-green-600 bg-green-50 px-2.5 py-1 rounded-
full text-sm font-medium">
                        <CheckCircle size={16} /> Active
                    </div>
                </div>
                <div className="flex items-center justify-between">
                    <span className="text-slate-600 font-medium">MFA</span>
                    <div className="flex items-center gap-1.5 text-green-600 bg-green-50 px-2.5 py-1 rounded-
full text-sm font-medium">
                        <CheckCircle size={16} /> Enabled
                    </div>
                </div>
                <div className="flex items-center justify-between">
                    <span className="text-slate-600 font-medium">Data Encryption</span>
                    <div className="flex items-center gap-1.5 text-green-600 bg-green-50 px-2.5 py-1 rounded-
full text-sm font-medium">
                        <CheckCircle size={16} /> AES-256
                    </div>
                </div>
                <div className="flex items-center justify-between">
                    <span className="text-slate-600 font-medium">Blockchain Minting</span>
                    <div className="flex items-center gap-1.5 text-green-600 bg-green-50 px-2.5 py-1 rounded-
full text-sm font-medium">
                        <CheckCircle size={16} /> Connected
                    </div>
                </div>
            </div>
        </div>

        <div className="lg:col-span-3 bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <h3 className="text-lg font-semibold text-slate-800 mb-4">Recent Activity</h3>
            <div className="overflow-x-auto">
                <table className="w-full text-sm text-left">
                    <thead className="text-xs text-slate-500 uppercase bg-slate-50">
                        <tr>
                            <th className="px-4 py-3 rounded-tl-lg">Time</th>
                            <th className="px-4 py-3">User</th>
                            <th className="px-4 py-3">Action</th>
                        </tr>
                    </thead>
                </table>
            </div>
        </div>
    </div>

```

```

        <th className="px-4 py-3 rounded-tr-lg">Target</th>
      </tr>
    </thead>
    <tbody>
      {recentLogs.length > 0 ? recentLogs.map((log, i) => (
        <tr key={i} className="border-b border-slate-100 last:border-0">
          <td className="px-4 py-3 text-slate-600">{new
Date(log.timestamp).toLocaleString()}</td>
          <td className="px-4 py-3 font-medium text-slate-800">{log.user || 'System'}</td>
          <td className="px-4 py-3">
            <span className="bg-slate-100 text-slate-700 px-2 py-1 rounded text-xs font-medium">
              {log.action}
            </span>
          </td>
          <td className="px-4 py-3 text-slate-600">{log.target || '-'}</td>
        </tr>
      )) : (
        <tr>
          <td colspan="4" className="px-4 py-8 text-center text-slate-500">No recent
activity</td>
        </tr>
      )}
    </tbody>
  </table>
</div>
</div>
</div>
</div>
);
};

export default Dashboard;

```

Kode 25. WAF Dashboard & Simulator (frontend/src/pages/WafDashboard.jsx)

```

import React, { useState, useEffect } from 'react';
import api from '../utils/api';
import { Shield, ShieldAlert, Zap, Server, Activity, Database, Key } from 'lucide-react';
import { BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
PieChart, Pie, Cell } from 'recharts';

const WafDashboard = () => {
  const [wafStats, setWafStats] = useState(null);
  const [logs, setLogs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [testResult, setTestResult] = useState(null);

  useEffect(() => {
    fetchWafData();
  }, []);

  const fetchWafData = async () => {
    try {
      setLoading(true);
      const [statsRes, logsRes] = await Promise.all([
        api.get('/audit/waf/stats').catch(() => ({ data: { success: true, data: { totalBlocked: 0,
byAttackType: [], bySeverity: [] } } })),
        api.get('/audit/waf').catch(() => ({ data: { success: true, data: [] } } ))
      ]);

      const statsData = statsRes.data?.data || statsRes.data || {};
      const logsData = logsRes.data?.data || logsRes.data || [];
    }
  };

```

```

// Map stats to chart format
const byType = (statsData.byAttackType || []).map(item => ({
  name: item.attack_type === 'SQL_INJECTION' ? 'SQLi' : item.attack_type === 'XSS' ? 'XSS' :
item.attack_type === 'CSRF' ? 'CSRF' : item.attack_type === 'RATE_LIMIT' ? 'Rate Limit' :
item.attack_type,
  count: item.count
}));

setWafStats({ totalBlocked: statsData.totalBlocked || 0, byType });
setLogs(Array.isArray(logsData) ? logsData : []);
} catch (err) {
  console.error(err);
} finally {
  setLoading(false);
}
};

const handleTestAttack = async (type) => {
  setTestResult({ status: 'LOADING', message: 'Mengirim payload...' });
  let payload = {};

  if (type === 'SQLI') {
    payload = { username: "admin' OR '1'='1' --", password: "pwd" };
  } else if (type === 'XSS') {
    payload = { username: "<script>alert('XSS')</script>", password: "pwd" };
  } else if (type === 'RATE') {
    // Send multiple rapid requests to trigger rate limit
    const promises = Array.from({ length: 10 }, () =>
      fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ username: "test", password: "test" })
      })
    );
  });
  const results = await Promise.all(promises);
  const blocked = results.some(r => r.status === 429);
  setTestResult({
    status: blocked ? 'BLOCKED' : 'ALLOWED',
    type: 'RATE_LIMIT',
    message: blocked
      ? '🚫 WAF berhasil memblokir! Terlalu banyak request dari IP yang sama.'
      : 'Batas rate limit belum tercapai, coba lagi.'
  });
  setTimeout(fetchWafData, 1500);
  return;
}

try {
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });

  if (res.status === 403) {
    const data = await res.json();
    setTestResult({
      status: 'BLOCKED',
      type: data.attack_type || type,
      message: `🚫 WAF berhasil memblokir serangan ${type === 'SQLI' ? 'SQL Injection' : 'XSS'}!
Payload berbahaya terdeteksi dan diblokir.`
    });
  }
}

```



```

    } else if (res.status === 429) {
      setTestResult({
        status: 'BLOCKED',
        type: 'RATE_LIMIT',
        message: '🚫 WAF berhasil memblokir! Terlalu banyak request dari IP yang sama.'
      });
    } else {
      setTestResult({
        status: 'ALLOWED',
        type: type,
        message: 'Request berhasil melewati WAF. Coba lagi atau periksa server.'
      });
    }
  } catch (err) {
    setTestResult({
      status: 'BLOCKED',
      type: type,
      message: `🚫 WAF berhasil memblokir serangan ${type}! Koneksi diputus oleh WAF.`
    });
  }
  setTimeout(fetchWafData, 1500);
};

const COLORS = ['#ef4444', '#f97316', '#eab308', '#3b82f6', '#8b5cf6'];

return (
  <div className="p-8">
    <div className="flex items-center gap-3 mb-8">
      <ShieldAlert className="w-8 h-8 text-red-500" />
      <h1 className="text-2xl font-bold text-slate-800">WAF Dashboard - Web Application Firewall</h1>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-4 gap-6 mb-8">
      <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6 border-l-4 border-l-red-500">
        <p className="text-sm font-medium text-slate-500 mb-1">Total Attacks Blocked</p>
        <h3 className="text-3xl font-bold text-slate-800">{wafStats?.totalBlocked || 0}</h3>
      </div>
      <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6 border-l-4 border-l-orange-500">
        <p className="text-sm font-medium text-slate-500 mb-1">SQL Injection Attempts</p>
        <h3 className="text-3xl font-bold text-slate-800">
          {wafStats?.byType?.find(t => t.name === 'SQLi')?.count || 0}
        </h3>
      </div>
      <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6 border-l-4 border-l-yellow-500">
        <p className="text-sm font-medium text-slate-500 mb-1">XSS Attempts</p>
        <h3 className="text-3xl font-bold text-slate-800">
          {wafStats?.byType?.find(t => t.name === 'XSS')?.count || 0}
        </h3>
      </div>
      <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6 border-l-4 border-l-blue-500">
        <p className="text-sm font-medium text-slate-500 mb-1">Rate Limit Violations</p>
        <h3 className="text-3xl font-bold text-slate-800">
          {wafStats?.byType?.find(t => t.name === 'Rate Limit')?.count || 0}
        </h3>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-8">
      <div className="lg:col-span-2 bg-white rounded-xl shadow-sm border border-slate-200 p-6">

```

```

<h3 className="text-lg font-semibold text-slate-800 mb-4">Attacks by Type</h3>
<div className="h-72">
  <ResponsiveContainer width="100%" height="100%">
    <BarChart data={wafStats?.byType || []}>
      <CartesianGrid strokeDasharray="3 3" vertical={false} />
      <XAxis dataKey="name" />
      <YAxis />
      <Tooltip />
      <Bar dataKey="count" fill="#3b82f6" radius={[4, 4, 0, 0]} />
    </BarChart>
  </ResponsiveContainer>
</div>
</div>
<div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
  <h3 className="text-lg font-semibold text-slate-800 mb-4">WAF Rules Status</h3>
  <div className="space-y-4">
    <div className="flex justify-between items-center p-3 bg-slate-50 rounded-lg">
      <span className="flex items-center gap-2 font-medium text-slate-700"><Database size={16}/>
      SQL Injection Filter</span>
      <span className="text-green-600 bg-green-100 px-2 py-1 rounded text-xs font-bold">✔️</span>
    </div>
    <div className="flex justify-between items-center p-3 bg-slate-50 rounded-lg">
      <span className="flex items-center gap-2 font-medium text-slate-700"><Zap size={16}/> XSS
      Filter</span>
      <span className="text-green-600 bg-green-100 px-2 py-1 rounded text-xs font-bold">✔️</span>
    </div>
    <div className="flex justify-between items-center p-3 bg-slate-50 rounded-lg">
      <span className="flex items-center gap-2 font-medium text-slate-700"><Activity size={16}/>
      Rate Limiting</span>
      <span className="text-green-600 bg-green-100 px-2 py-1 rounded text-xs font-bold">✔️ 100
      req/min</span>
    </div>
    <div className="flex justify-between items-center p-3 bg-slate-50 rounded-lg">
      <span className="flex items-center gap-2 font-medium text-slate-700"><Key size={16}/> CSRF
      Protection</span>
      <span className="text-green-600 bg-green-100 px-2 py-1 rounded text-xs font-bold">✔️</span>
    </div>
    <div className="flex justify-between items-center p-3 bg-slate-50 rounded-lg">
      <span className="flex items-center gap-2 font-medium text-slate-700"><Server size={16}/>
      DDoS Protection</span>
      <span className="text-green-600 bg-green-100 px-2 py-1 rounded text-xs font-bold">✔️</span>
    </div>
  </div>
</div>
</div>
<div className="grid grid-cols-1 lg:grid-cols-4 gap-6 mb-8">
  <div className="lg:col-span-3 bg-white rounded-xl shadow-sm border border-slate-200 p-6">
    <h3 className="text-lg font-semibold text-slate-800 mb-4">Recent Blocked Requests</h3>
    <div className="overflow-x-auto">
      <table className="w-full text-sm text-left">
        <thead className="text-xs text-slate-500 uppercase bg-slate-50">
          <tr>
            <th className="px-4 py-3">Time</th>
            <th className="px-4 py-3">IP Address</th>
            <th className="px-4 py-3">Path</th>
            <th className="px-4 py-3">Attack Type</th>
            <th className="px-4 py-3">Severity</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>
          </tr>
          <tr>
            <td>2023-10-27 10:15:30</td>
            <td>192.168.1.1</td>
            <td>/api/v1/users</td>
            <td>SQL Injection</td>
            <td>High</td>

```

```

    </thead>
    <tbody>
      {logs.length > 0 ? logs.map((log, i) => (
        <tr key={i} className="border-b border-slate-100 font-mono text-xs">
          <td className="px-4 py-3 text-slate-600">{new
Date(log.created_at).toLocaleString()}</td>
          <td className="px-4 py-3">{log.ip_address}</td>
          <td className="px-4 py-3 text-slate-500">{log.method} {log.path}</td>
          <td className="px-4 py-3 font-semibold text-red-600">{log.attack_type}</td>
          <td className="px-4 py-3">
            <span className={`px-2 py-1 rounded text-white ${log.severity === 'CRITICAL' ? 'bg-
red-600' : log.severity === 'HIGH' ? 'bg-orange-500' : 'bg-yellow-500'}`}>
              {log.severity || 'HIGH'}
            </span>
          </td>
        </tr>
      )) : (
        <tr><td colspan="5" className="text-center py-6 text-slate-500">No blocked requests
recently.</td></tr>
      )}
    </tbody>
  </table>
</div>
</div>
<div className="bg-slate-900 rounded-xl shadow-sm p-6 text-white border border-slate-700">
  <h3 className="text-lg font-semibold mb-4 flex items-center gap-2"><ShieldAlert size={20}/>
WAF Simulator</h3>
  <p className="text-slate-400 text-sm mb-6">Test the WAF by sending malicious payloads. The WAF
should intercept and block these requests.</p>

  <div className="space-y-3">
    <button onClick={() => handleTestAttack('SQLI')} className="w-full bg-slate-800 hover:bg-
slate-700 border border-slate-600 py-2 px-4 rounded text-left text-sm font-mono text-orange-400
transition">
      {">"} Test SQL Injection
    </button>
    <button onClick={() => handleTestAttack('XSS')} className="w-full bg-slate-800 hover:bg-
slate-700 border border-slate-600 py-2 px-4 rounded text-left text-sm font-mono text-yellow-400
transition">
      {">"} Test XSS
    </button>
    <button onClick={() => handleTestAttack('RATE')} className="w-full bg-slate-800 hover:bg-
slate-700 border border-slate-600 py-2 px-4 rounded text-left text-sm font-mono text-blue-400 transition">
      {">"} Test Rate Limit
    </button>
  </div>

  {testResult && testResult.status !== 'LOADING' && (
    <div className={`mt-6 p-4 rounded-lg border ${testResult.status === 'BLOCKED' ? 'bg-red-600
border-red-400 text-white' : 'bg-green-800 border-green-500 text-green-200'}`}>
      <p className="text-sm font-bold font-mono">
        {testResult.status === 'BLOCKED' ? '🔥 SERANGAN TERDETEKSI & DIBLOKIR!' : '🚀 Request
Lolos dari WAF'}
      </p>
      <p className="text-sm font-medium font-mono mt-1">{testResult.message}</p>
    </div>
  )}
  {testResult && testResult.status === 'LOADING' && (
    <div className="mt-6 p-4 rounded-lg border bg-slate-700 border-slate-500 text-slate-300">
      <p className="text-sm font-mono animate-pulse">⌚ {testResult.message}</p>
    </div>
  )}
</div>

```

```

        </div>
      </div>
    );
  };

export default WafDashboard;

```

Kode 26. Manajemen Data Siswa (frontend/src/pages/Students.jsx)

```

import React, { useState, useEffect } from 'react';
import api from '../utils/api';
import { Search, Plus, Edit2, Trash2, Lock, X } from 'lucide-react';

const Students = () => {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [currentStudent, setCurrentStudent] = useState(null);

  const [formData, setFormData] = useState({
    nis: '', nisp: '', nama: '', tempatLahir: '', tanggalLahir: '',
    jenisKelamin: 'L', kelas: '', jurusan: '', tahunMasuk: '',
    nik: '', alamat: '', noHp: ''
  });

  const fetchStudents = async () => {
    try {
      setLoading(true);
      const res = await api.get('/students');
      setStudents(res.data.data || []);
    } catch (error) {
      console.error('Error fetching students:', error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchStudents();
  }, []);

  const handleInputChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const openModal = (student = null) => {
    if (student) {
      setCurrentStudent(student);
      setFormData(student);
    } else {
      setCurrentStudent(null);
      setFormData({
        nis: '', nisp: '', nama: '', tempatLahir: '', tanggalLahir: '',
        jenisKelamin: 'L', kelas: '', jurusan: '', tahunMasuk: '',
        nik: '', alamat: '', noHp: ''
      });
    }
    setIsModalOpen(true);
  };

  const closeModal = () => {

```

```

    setIsModalOpen(false);
  };

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    if (currentStudent) {
      await api.put(`/students/${currentStudent.id || currentStudent._id}`, formData);
    } else {
      await api.post('/students', formData);
    }
    closeModal();
    fetchStudents();
  } catch (error) {
    console.error('Error saving student:', error);
    alert('Gagal menyimpan data siswa');
  }
};

const handleDelete = async (id) => {
  if (window.confirm('Yakin ingin menghapus siswa ini?')) {
    try {
      await api.delete(`/students/${id}`);
      fetchStudents();
    } catch (error) {
      console.error('Error deleting student:', error);
    }
  }
};

const filteredStudents = students.filter(s =>
  (s.nama || '').toLowerCase().includes(search.toLowerCase()) ||
  (s.nis || '').includes(search)
);

return (
  <div className="p-8">
    <div className="flex justify-between items-center mb-6">
      <h1 className="text-2xl font-bold text-slate-800">Manajemen Data Siswa</h1>
      <button
        onClick={() => openModal()}
        className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg flex items-center gap-2
transition"
      >
        <Plus size={18} /> Tambah Siswa
      </button>
    </div>

    <div className="bg-white rounded-xl shadow-sm border border-slate-200 overflow-hidden">
      <div className="p-4 border-b border-slate-100 flex gap-4">
        <div className="relative flex-1 max-w-md">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-5 h-5" />
          <input
            type="text"
            placeholder="Cari berdasarkan Nama atau NIS..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-slate-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-blue-500 outline-none"
          />
        </div>
      </div>
    </div>
  </div>

```

```

<div className="overflow-x-auto">
  <table className="w-full text-sm text-left">
    <thead className="text-xs text-slate-500 uppercase bg-slate-50">
      <tr>
        <th className="px-6 py-3">No</th>
        <th className="px-6 py-3">NIS</th>
        <th className="px-6 py-3">Nama Lengkap</th>
        <th className="px-6 py-3">Kelas</th>
        <th className="px-6 py-3">Jurusan</th>
        <th className="px-6 py-3 text-right">Aksi</th>
      </tr>
    </thead>
    <tbody>
      {loading ? (
        <tr><td colspan="6" className="text-center py-8 text-slate-500">Loading...</td></tr>
      ) : filteredStudents.length > 0 ? (
        filteredStudents.map((student, idx) => (
          <tr key={student.id || student._id || idx} className="border-b border-slate-100
hover:bg-slate-50">
            <td className="px-6 py-4">{idx + 1}</td>
            <td className="px-6 py-4 font-medium text-slate-900">{student.nis}</td>
            <td className="px-6 py-4">{student.nama}</td>
            <td className="px-6 py-4">{student.kelas}</td>
            <td className="px-6 py-4">{student.jurusan}</td>
            <td className="px-6 py-4 text-right">
              <div className="flex justify-end gap-2">
                <button onClick={() => openModal(student)} className="p-1.5 text-blue-600
hover:bg-blue-50 rounded">
                  <Edit2 size={16} />
                </button>
                <button onClick={() => handleDelete(student.id || student._id)} className="p-1.5
text-red-600 hover:bg-red-50 rounded">
                  <Trash2 size={16} />
                </button>
              </div>
            </td>
          </tr>
        ))
      ) : (
        <tr><td colspan="6" className="text-center py-8 text-slate-500">Tidak ada data
siswa.</td></tr>
      )}
    </tbody>
  </table>
</div>
</div>

{isModalOpen && (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-xl shadow-xl w-full max-w-2xl max-h-[90vh] flex flex-col">
      <div className="flex justify-between items-center p-6 border-b border-slate-100">
        <h2 className="text-xl font-bold text-slate-800">{currentStudent ? 'Edit Data Siswa' :
'Tambah Siswa Baru'}</h2>
        <button onClick={closeModal} className="text-slate-400 hover:text-slate-600">
          <X size={24} />
        </button>
      </div>
      <div className="p-6 overflow-y-auto">
        <form id="studentForm" onSubmit={handleSubmit} className="grid grid-cols-1 md:grid-cols-2
gap-4">
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">NIS</label>

```

```

        <input type="text" name="nis" value={formData.nis} onChange={handleInputChange} required
className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">NISN</label>
        <input type="text" name="nisn" value={formData.nisn} onChange={handleInputChange}
required className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div className="md:col-span-2">
        <label className="block text-sm font-medium text-slate-700 mb-1">Nama Lengkap</label>
        <input type="text" name="nama" value={formData.nama} onChange={handleInputChange}
required className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">Tempat Lahir</label>
        <input type="text" name="tempatLahir" value={formData.tempatLahir}
onChange={handleInputChange} className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500
outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">Tanggal Lahir</label>
        <input type="date" name="tanggalLahir" value={formData.tanggalLahir}
onChange={handleInputChange} className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500
outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">Kelas</label>
        <input type="text" name="kelas" value={formData.kelas} onChange={handleInputChange}
required className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">Jurusan</label>
        <input type="text" name="jurusan" value={formData.jurusan} onChange={handleInputChange}
required className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>

    <div className="md:col-span-2 mt-4 mb-2">
        <h3 className="text-sm font-semibold text-slate-600 flex items-center gap-2 border-b pb-
2"><Lock size={16} className="text-blue-500"/> Data Sensitif (Dienkripsi)</h3>
    </div>

    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1 flex items-center gap-
1">NIK <Lock size={12} className="text-slate-400"/></label>
        <input type="text" name="nik" value={formData.nik} onChange={handleInputChange}
className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-1 flex items-center gap-
1">No HP <Lock size={12} className="text-slate-400"/></label>
        <input type="text" name="noHp" value={formData.noHp} onChange={handleInputChange}
className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
    <div className="md:col-span-2">
        <label className="block text-sm font-medium text-slate-700 mb-1 flex items-center gap-
1">Alamat Lengkap <Lock size={12} className="text-slate-400"/></label>
        <textarea name="alamat" value={formData.alamat} onChange={handleInputChange} rows="2"
className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none" />
    </div>
</form>
</div>

```

```

        <div className="p-6 border-t border-slate-100 flex justify-end gap-3 bg-slate-50 rounded-b-
xl">
            <button onClick={closeModal} className="px-4 py-2 text-slate-600 bg-white border border-
slate-300 rounded-lg hover:bg-slate-50 transition">Batal</button>
            <button type="submit" form="studentForm" className="px-4 py-2 bg-blue-600 text-white
rounded-lg hover:bg-blue-700 transition">Simpan Data</button>
        </div>
    </div>
</div>
)}
</div>
);
};

export default Students;

```

Kode 27. Sertifikat & Ijazah (frontend/src/pages/Certificates.jsx)

```

import React, { useState, useEffect } from 'react';
import api from '../utils/api';
import { useAuth } from '../contexts/AuthContext';
import { Award, Upload, Link as LinkIcon, CheckCircle, AlertTriangle, ShieldAlert, X } from 'lucide-
react';

const Certificates = () => {
    const [certificates, setCertificates] = useState([]);
    const [students, setStudents] = useState([]);
    const [loading, setLoading] = useState(true);
    const [isModalOpen, setIsModalOpen] = useState(false);
    const { user } = useAuth();

    const [formData, setFormData] = useState({ student_id: '', type: 'IJAZAH' });
    const [file, setFile] = useState(null);

    const fetchCerts = async () => {
        try {
            setLoading(true);
            const res = await api.get('/certificates');
            setCertificates(res.data.data || []);
        } catch (e) {
            console.error(e);
        } finally {
            setLoading(false);
        }
    };

    const fetchStudents = async () => {
        try {
            const res = await api.get('/students');
            setStudents(res.data.data || []);
        } catch (e) {
            console.error(e);
        }
    };

    useEffect(() => {
        fetchCerts();
        fetchStudents();
    }, []);

    const handleFileChange = (e) => {
        setFile(e.target.files[0]);
    };

```



```

};

const handleSubmit = async (e) => {
  e.preventDefault();
  if (!file) return alert("Pilih file PDF!");

  const data = new FormData();
  data.append('student_id', formData.student_id);
  data.append('type', formData.type);
  data.append('pdf', file);

  try {
    await api.post('/certificates/issue', data);
    setIsModalOpen(false);
    setFile(null);
    fetchCerts();
    alert('Sertifikat berhasil dibuat! Menunggu persetujuan Kepala Sekolah.');
```

```

  } catch (e) {
    alert('Gagal membuat sertifikat: ' + (e.response?.data?.error || e.message));
  }
};

const handleApprove = async (id) => {
  try {
    await api.post(`/certificates/${id}/approve`);
    fetchCerts();
    alert('Sertifikat berhasil disetujui dan disimpan ke Blockchain!');
```

```

  } catch (e) {
    alert('Gagal menyetujui sertifikat: ' + (e.response?.data?.error || e.message));
  }
};

return (
  <div className="p-8">
    <div className="flex justify-between items-center mb-6">
      <h1 className="text-2xl font-bold text-slate-800">Penerbitan Sertifikat & Ijazah Digital</h1>
      {(user?.role === 'TU' || user?.role === 'ADMIN') && (
        <button onClick={() => setIsModalOpen(true)} className="bg-blue-600 hover:bg-blue-700 text-white
px-4 py-2 rounded-lg flex items-center gap-2">
          <Upload size={18} /> Terbitkan Sertifikat
        </button>
      )}
    </div>

    <div className="bg-white rounded-xl shadow-sm border border-slate-200 overflow-hidden">
      <div className="overflow-x-auto">
        <table className="w-full text-sm text-left">
          <thead className="text-xs text-slate-500 uppercase bg-slate-50">
            <tr>
              <th className="px-6 py-3">No. Sertifikat</th>
              <th className="px-6 py-3">Nama Siswa</th>
              <th className="px-6 py-3">Tipe</th>
              <th className="px-6 py-3">Status</th>
              <th className="px-6 py-3">Blockchain Tx</th>
              <th className="px-6 py-3 text-right">Aksi</th>
            </tr>
          </thead>
          <tbody>
            {loading ? (
              <tr><td colspan="6" className="text-center py-8">Loading...</td></tr>
            ) : certificates.length > 0 ? (
              certificates.map((cert) => (
                <tr key={cert.id} className="border-b hover:bg-slate-50">

```

```

        <td className="px-6 py-4 font-medium">{cert.certificate_number}</td>
        <td className="px-6 py-4">{cert.student_name}</td>
        <td className="px-6 py-4">{cert.type}</td>
        <td className="px-6 py-4">
          <span className={`px-2.5 py-0.5 rounded text-xs font-medium ${cert.status ===
'PENDING' ? 'bg-yellow-100 text-yellow-800' : 'bg-green-100 text-green-800'}`}>
            {cert.status}
          </span>
        </td>
        <td className="px-6 py-4">
          {cert.blockchain_tx_hash ? (
            <span className="text-blue-600 flex items-center gap-1 font-mono text-xs">
              <LinkIcon size={14} /> {cert.blockchain_tx_hash.substring(0, 15) + '...'}
            </span>
          ) : '-'}
        </td>
        <td className="px-6 py-4 text-right">
          {cert.status === 'PENDING' && user?.role === 'KEPALA_SEKOLAH' && (
            <button onClick={() => handleApprove(cert.id)} className="text-white bg-green-600
hover:bg-green-700 px-3 py-1 rounded-md text-xs font-medium mr-2">
              Approve
            </button>
          )}
        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>

{isModalOpen && (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-xl shadow-xl w-full max-w-md flex flex-col">
      <div className="flex justify-between items-center p-6 border-b border-slate-100">
        <h2 className="text-xl font-bold text-slate-800">Terbitkan Sertifikat Baru</h2>
        <button onClick={() => setIsModalOpen(false)} className="text-slate-400 hover:text-slate-
600">
          <X size={24} />
        </button>
      </div>
      <form onSubmit={handleSubmit} className="p-6">
        <div className="mb-4">
          <label className="block text-sm font-medium text-slate-700 mb-1">Nama Siswa</label>
          <select
            required
            value={formData.student_id}
            onChange={(e) => setFormData({...formData, student_id: e.target.value})}
            className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none"
          >
            <option value="">Pilih Siswa</option>
            {students.map(s => (
              <option key={s.id} value={s.id}>{s.full_name} ({s.nis})</option>
            ))}
          </select>
        </div>
        <div className="mb-4">

```

```

        <label className="block text-sm font-medium text-slate-700 mb-1">Tipe Sertifikat</label>
        <select
          value={formData.type}
          onChange={(e) => setFormData({...formData, type: e.target.value})}
          className="w-full p-2 border rounded-lg focus:ring-2 focus:ring-blue-500 outline-none"
        >
          <option value="IJAZAH">Ijazah</option>
          <option value="TRANSKRIP">Transkrip</option>
          <option value="SERTIFIKAT_PKL">Sertifikat PKL</option>
        </select>
      </div>

      <div className="mb-6">
        <label className="block text-sm font-medium text-slate-700 mb-1">Upload File PDF</label>
        <input
          type="file"
          accept="application/pdf"
          required
          onChange={handleFileChange}
          className="w-full text-sm text-slate-500 file:mr-4 file:py-2 file:px-4 file:rounded-lg
file:border-0 file:text-sm file:font-semibold file:bg-blue-50 file:text-blue-700 hover:file:bg-blue-100"
        />
      </div>

      <div className="flex justify-end gap-3 pt-4 border-t border-slate-100">
        <button type="button" onClick={() => setIsModalOpen(false)} className="px-4 py-2 text-
slate-600 bg-white border border-slate-300 rounded-lg hover:bg-slate-50">Batal</button>
        <button type="submit" className="px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-
blue-700">Simpan</button>
      </div>
    </form>
  </div>
</div>
)}
</div>
);
};

export default Certificates;

```

Kode 28. Setup MFA / 2FA (frontend/src/pages/MfaSetup.jsx)

```

import React, { useState } from 'react';
import api from '../utils/api';
import { ShieldCheck, Smartphone, CheckCircle } from 'lucide-react';

const MfaSetup = () => {
  const [setupData, setSetupData] = useState(null);
  const [token, setToken] = useState('');
  const [message, setMessage] = useState('');

  const handleEnableMfa = async () => {
    try {
      const res = await api.post('/auth/mfa/setup');
      setSetupData(res.data.data || res.data);
    } catch (err) {
      setMessage('Failed to initiate MFA setup.');

```

```

try {
  // Step 1: Enable MFA (this verifies the token AND enables it)
  await api.post('/auth/mfa/enable', { mfaToken: token });
  setMessage('MFA berhasil diaktifkan! Akun Anda sekarang dilindungi oleh Autentikasi Dua Langkah.');
```

```

  setSetupData(null);
} catch (err) {
  setMessage('Token tidak valid. Pastikan kode 6 digit dari aplikasi Authenticator Anda benar.');
```

```

}
};

return (
  <div className="p-8 max-w-3xl mx-auto">
    <div className="flex items-center gap-3 mb-8">
      <ShieldCheck className="w-8 h-8 text-blue-600" />
      <h1 className="text-2xl font-bold text-slate-800">Multi-Factor Authentication (MFA) Setup</h1>
    </div>

    <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-8">
      {message && (
        <div className="mb-6 p-4 bg-slate-100 text-slate-800 rounded-lg flex items-center gap-2">
          <CheckCircle className="text-green-500 w-5 h-5"/>
          {message}
        </div>
      )}

      {!setupData ? (
        <div className="text-center py-8">
          <Smartphone className="w-16 h-16 text-slate-400 mx-auto mb-4" />
          <h2 className="text-xl font-bold text-slate-800 mb-2">Secure Your Account</h2>
          <p className="text-slate-600 mb-6">Add an extra layer of security to your account by enabling
Multi-Factor Authentication.</p>
          <button
            onClick={handleEnableMfa}
            className="bg-blue-600 hover:bg-blue-700 text-white font-bold py-2.5 px-6 rounded-lg
transition"
          >
            Setup Authenticator App
          </button>
        </div>
      ) : (
        <div className="space-y-6">
          <div className="flex flex-col md:flex-row gap-8 items-center">
            <div className="bg-white p-4 border border-slate-200 rounded-xl shadow-sm">
              <img src={setupData.qrcode} alt="QR Code" className="w-48 h-48" />
            </div>
            <div className="flex-1 space-y-4">
              <h3 className="text-lg font-bold text-slate-800">Step 1: Scan QR Code</h3>
              <p className="text-slate-600 text-sm">Open Google Authenticator, Authy, or your preferred
authenticator app and scan the QR code.</p>
              <div className="pt-2">
                <span className="text-xs text-slate-500 uppercase font-semibold">Or enter manual
key:</span>
                <div className="mt-1 font-mono text-sm bg-slate-100 p-2 rounded border border-slate-200
break-all">
                  {setupData.secret}
                </div>
              </div>
            </div>
          </div>

          <hr className="border-slate-100" />

          <div className="space-y-4">

```

```

        <h3 className="text-lg font-bold text-slate-800">Step 2: Verify Code</h3>
        <p className="text-slate-600 text-sm">Enter the 6-digit code generated by your authenticator
app to confirm setup.</p>

        <form onSubmit={handleVerify} className="flex gap-4">
            <input
                type="text"
                value={token}
                onChange={(e) => setToken(e.target.value)}
                placeholder="000000"
                className="flex-1 max-w-xs p-3 text-lg tracking-widest font-mono border border-slate-300
rounded-lg focus:ring-2 focus:ring-blue-500 outline-none"
                maxLength="6"
                required
            />
            <button type="submit" className="bg-slate-900 hover:bg-slate-800 text-white font-bold py-3
px-6 rounded-lg transition">
                Verify & Enable
            </button>
        </form>
    </div>
</div>
    )}
</div>
</div>
);
};

export default MfaSetup;

```

Kode 29. Audit Log (frontend/src/pages/AuditLog.jsx)

```

import React, { useState, useEffect } from 'react';
import api from '../utils/api';
import { ClipboardList, Download } from 'lucide-react';

const AuditLog = () => {
    const [logs, setLogs] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const fetchLogs = async () => {
            try {
                const res = await api.get('/audit/logs');
                setLogs(res.data);
            } catch (err) {
                console.error(err);
            } finally {
                setLoading(false);
            }
        };
        fetchLogs();
    }, []);

    return (
        <div className="p-8">
            <div className="flex justify-between items-center mb-6">
                <div className="flex items-center gap-3">
                    <ClipboardList className="w-8 h-8 text-blue-600" />
                    <h1 className="text-2xl font-bold text-slate-800">Audit Log & Security Events</h1>
                </div>
            </div>

```

```

        <button className="bg-slate-800 hover:bg-slate-900 text-white px-4 py-2 rounded-lg flex items-center gap-2 transition">
          <Download size={18} /> Export CSV
        </button>
      </div>

      <div className="bg-white rounded-xl shadow-sm border border-slate-200 overflow-hidden">
        <div className="overflow-x-auto">
          <table className="w-full text-sm text-left">
            <thead className="text-xs text-slate-500 uppercase bg-slate-50">
              <tr>
                <th className="px-6 py-3">Timestamp</th>
                <th className="px-6 py-3">User</th>
                <th className="px-6 py-3">Action</th>
                <th className="px-6 py-3">Target</th>
                <th className="px-6 py-3">IP Address</th>
              </tr>
            </thead>
            <tbody>
              {loading ? (
                <tr><td colspan="5" className="text-center py-8">Loading logs...</td></tr>
              ) : logs.length > 0 ? (
                logs.map((log, i) => (
                  <tr key={i} className="border-b border-slate-100">
                    <td className="px-6 py-4 text-slate-600 font-mono text-xs">{new
Date(log.timestamp).toLocaleString()}</td>
                    <td className="px-6 py-4 font-medium">{log.user || 'System'}</td>
                    <td className="px-6 py-4">
                      <span className="bg-slate-100 text-slate-700 px-2 py-1 rounded text-xs font-semibold
uppercase">{log.action}</span>
                    </td>
                    <td className="px-6 py-4 text-slate-600">{log.target || '-'}</td>
                    <td className="px-6 py-4 text-slate-500 font-mono text-xs">{log.ip || '-'}</td>
                  </tr>
                ))
              ) : (
                <tr><td colspan="5" className="text-center py-8 text-slate-500">No audit logs
found.</td></tr>
              )}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
};

export default AuditLog;

```

Kode 30. Manajemen Nilai (frontend/src/pages/Grades.jsx)

```

import React from 'react';

const Grades = () => {
  return (
    <div className="p-8">
      <h1 className="text-2xl font-bold text-slate-800 mb-6">Manajemen Nilai Siswa</h1>
      <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-8 text-center text-slate-500">
        <p>Fitur Manajemen Nilai sedang dalam tahap pengembangan.</p>
      </div>
    </div>
  );
};

```

```

    );
  });

export default Grades;

```

Kode 31. Portal Verifikasi Publik (frontend/src/pages/VerifyDocument.jsx)

```

import React, { useState } from 'react';
import axios from 'axios';
import { Upload, FileText, CheckCircle, XCircle, AlertTriangle, Shield } from 'lucide-react';

const VerifyDocument = () => {
  const [file, setFile] = useState(null);
  const [hashInput, setHashInput] = useState('');
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  async function hashFile(file) {
    const buffer = await file.arrayBuffer();
    const hashBuffer = await crypto.subtle.digest('SHA-256', buffer);
    const hashArray = Array.from(new Uint8Array(hashBuffer));
    return hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
  }

  const handleVerify = async (e) => {
    e.preventDefault();
    if (!file && !hashInput) {
      setError('Silakan upload file PDF atau masukkan hash dokumen.');
```

return;

```

    }

    setError('');
    setLoading(true);
    setResult(null);

    try {
      let hashToVerify = hashInput;

      if (file) {
        hashToVerify = await hashFile(file);
      }

      // We use raw axios here since this is public and doesn't need auth token
      const res = await axios.post('/api/certificates/verify', { hash: hashToVerify });
      setResult({ status: 'VALID', data: res.data });
    } catch (err) {
      if (err.response && err.response.status === 404) {
        setResult({ status: 'INVALID', message: 'Dokumen tidak ditemukan di blockchain atau telah dimanipulasi.' });
      } else if (err.response && err.response.status === 410) {
        setResult({ status: 'REVOKED', message: err.response.data.message || 'Sertifikat telah dicabut.', reason: err.response.data.reason });
      } else {
        setError('Terjadi kesalahan saat memverifikasi dokumen.');
```

}

```

    } finally {
      setLoading(false);
    }
  }
};

return (

```

```

<div className="min-h-screen bg-slate-50 flex flex-col items-center py-12 px-4 sm:px-6 lg:px-8">
  <div className="max-w-3xl w-full text-center mb-8">
    <Shield className="mx-auto h-16 w-16 text-blue-600 mb-4" />
    <h1 className="text-3xl font-extrabold text-slate-900">🔒 Portal Verifikasi Dokumen SMK</h1>
    <p className="mt-4 text-lg text-slate-600">Verifikasi keaslian ijazah, transkrip, dan sertifikat
menggunakan teknologi Blockchain.</p>
  </div>

  <div className="w-full max-w-2xl bg-white rounded-xl shadow-lg p-8">
    {error && (
      <div className="mb-6 p-4 bg-red-50 text-red-700 rounded-lg flex items-center gap-3">
        <AlertTriangle className="w-5 h-5 flex-shrink-0" />
        <p>{error}</p>
      </div>
    )}

    <form onSubmit={handleVerify} className="space-y-6">
      <div className="border-2 border-dashed border-slate-300 rounded-xl p-8 text-center hover:border-
blue-500 hover:bg-blue-50 transition cursor-pointer">
        <input
          type="file"
          accept=".pdf"
          className="hidden"
          id="file-upload"
          onChange={(e) => setFile(e.target.files[0])}>
        />
        <label htmlFor="file-upload" className="cursor-pointer flex flex-col items-center">
          <Upload className="mx-auto h-12 w-12 text-slate-400" />
          <span className="mt-4 text-sm font-medium text-slate-900">
            {file ? file.name : 'Pilih File PDF atau Drag & Drop ke sini'}
          </span>
          <span className="mt-1 text-xs text-slate-500">Maksimal ukuran file: 5MB</span>
        </label>
      </div>

      <div className="relative">
        <div className="absolute inset-0 flex items-center">
          <div className="w-full border-t border-slate-300"></div>
        </div>
        <div className="relative flex justify-center text-sm">
          <span className="px-2 bg-white text-slate-500">Atau masukkan hash manual</span>
        </div>
      </div>

      <div>
        <input
          type="text"
          value={hashInput}
          onChange={(e) => setHashInput(e.target.value)}
          placeholder="Masukkan SHA-256 Hash dokumen"
          className="w-full px-4 py-3 border border-slate-300 rounded-lg focus:ring-2 focus:ring-blue-
500 outline-none">
        />
      </div>

      <button
        type="submit"
        disabled={loading}
        className="w-full bg-blue-600 hover:bg-blue-700 text-white font-bold py-3 px-4 rounded-lg
transition">
        >
        {loading ? 'Memverifikasi...' : 'Verifikasi Dokumen'}
      </button>

```



```

</form>

{result && (
  <div className="mt-8 pt-8 border-t border-slate-200">
    {result.status === 'VALID' && (
      <div className="bg-green-50 border border-green-200 rounded-xl p-6">
        <div className="flex items-center gap-4 mb-4">
          <CheckCircle className="w-10 h-10 text-green-500" />
          <div>
            <h3 className="text-lg font-bold text-green-800">Dokumen Valid (Terverifikasi)</h3>
            <p className="text-green-600 text-sm">Tercatat dalam blockchain dan tidak mengalami
perubahan.</p>
          </div>
        </div>
        <div className="bg-white rounded p-4 text-sm space-y-2">
          <div className="flex justify-between"><span className="text-slate-500">Nama:</span>
<span className="font-medium">{result.data.studentName || '-'}</span></div>
          <div className="flex justify-between"><span className="text-slate-500">Tipe:</span>
<span className="font-medium">{result.data.type || '-'}</span></div>
          <div className="flex justify-between"><span className="text-slate-500">Penerbit:</span>
<span className="font-medium">{result.data.issuer || 'SMK'}</span></div>
          <div className="flex justify-between"><span className="text-slate-500">Tx Hash:</span>
<span className="font-mono text-xs text-blue-600 break-all">{result.data.txHash || '-'}</span></div>
        </div>
      </div>
    )}

    {result.status === 'INVALID' && (
      <div className="bg-red-50 border border-red-200 rounded-xl p-6 text-center">
        <XCircle className="w-12 h-12 text-red-500 mx-auto mb-3" />
        <h3 className="text-lg font-bold text-red-800 mb-1">DOKUMEN TIDAK VALID</h3>
        <p className="text-red-600 text-sm">{result.message}</p>
      </div>
    )}

    {result.status === 'REVOKED' && (
      <div className="bg-orange-50 border border-orange-200 rounded-xl p-6 text-center">
        <AlertTriangle className="w-12 h-12 text-orange-500 mx-auto mb-3" />
        <h3 className="text-lg font-bold text-orange-800 mb-1">DOKUMEN DICABUT</h3>
        <p className="text-orange-600 text-sm mb-3">{result.message}</p>
        <div className="bg-white rounded p-3 text-sm text-left">
          <span className="font-medium text-orange-800">Alasan:</span> {result.reason || '-'}
        </div>
      </div>
    )}
  </div>
)}
</div>

<div className="mt-12 text-center text-sm text-slate-500">
  <p>Powered by Blockchain Technology & Zero Trust Architecture</p>
</div>
</div>
);
};

export default VerifyDocument;

```

BLOCKCHAIN - SMART CONTRACT

Kode 32. Smart Contract Sertifikat (blockchain/contracts/CertificateRegistry.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/**
 * @title CertificateRegistry
 * @dev Smart contract untuk manajemen sertifikat/ijazah SMK dengan blockchain.
 * Menyimpan hash dokumen secara immutable untuk verifikasi keaslian.
 * Menggunakan AccessControl untuk RBAC dan ReentrancyGuard untuk keamanan.
 */
contract CertificateRegistry is AccessControl, ReentrancyGuard {
    bytes32 public constant KEPALA_SEKOLAH_ROLE = keccak256("KEPALA_SEKOLAH_ROLE");
    bytes32 public constant TU_ROLE = keccak256("TU_ROLE");

    struct Certificate {
        string studentId;
        string certType;
        bytes32 documentHash;
        string metadata;
        address issuer;
        uint256 issuedAt;
        bool isRevoked;
        string revokeReason;
    }

    mapping(bytes32 => Certificate) public certificates;
    mapping(string => bytes32[]) public studentCertificates;

    uint256 public totalCertificates;
    bytes32[] public allCertificateHashes;

    event CertificateIssued(
        bytes32 indexed documentHash,
        string studentId,
        string certType,
        address issuer,
        uint256 timestamp
    );

    event CertificateRevoked(
        bytes32 indexed documentHash,
        string reason,
        address revokedBy,
        uint256 timestamp
    );

    event CertificateVerified(
        bytes32 indexed documentHash,
        bool isValid,
        address verifier,
        uint256 timestamp
    );

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(KEPALA_SEKOLAH_ROLE, msg.sender);
        _grantRole(TU_ROLE, msg.sender);
    }
}
```

```

}

/**
 * @dev Menerbitkan sertifikat baru. Hanya KEPALA_SEKOLAH atau ADMIN yang bisa.
 * @param studentId ID siswa
 * @param certType Tipe sertifikat (IJAZAH, TRANSKRIP, SERTIFIKAT_PKL, SERTIFIKAT_MAGANG)
 * @param documentHash Hash SHA-256 dari dokumen PDF
 * @param metadata JSON string berisi metadata tambahan
 */
function issueCertificate(
    string memory studentId,
    string memory certType,
    bytes32 documentHash,
    string memory metadata
) external nonReentrant {
    require(
        hasRole(KEPALA_SEKOLAH_ROLE, msg.sender) || hasRole(DEFAULT_ADMIN_ROLE, msg.sender),
        "CertificateRegistry: Must have KEPALA_SEKOLAH_ROLE or ADMIN_ROLE to issue"
    );
    require(certificates[documentHash].issuedAt == 0, "CertificateRegistry: Document hash already registered");
    require(bytes(studentId).length > 0, "CertificateRegistry: Student ID cannot be empty");
    require(bytes(certType).length > 0, "CertificateRegistry: Certificate type cannot be empty");

    certificates[documentHash] = Certificate({
        studentId: studentId,
        certType: certType,
        documentHash: documentHash,
        metadata: metadata,
        issuer: msg.sender,
        issuedAt: block.timestamp,
        isRevoked: false,
        revokeReason: ""
    });

    studentCertificates[studentId].push(documentHash);
    allCertificateHashes.push(documentHash);
    totalCertificates++;

    emit CertificateIssued(documentHash, studentId, certType, msg.sender, block.timestamp);
}

/**
 * @dev Verifikasi keaslian sertifikat berdasarkan hash dokumen.
 * Fungsi ini public – siapa saja bisa memverifikasi.
 */
function verifyCertificate(bytes32 documentHash) external view returns (
    bool isValid,
    string memory studentId,
    string memory certType,
    uint256 issuedAt,
    address issuer,
    bool isRevoked
) {
    Certificate memory cert = certificates[documentHash];

    if (cert.issuedAt == 0) {
        return (false, "", "", 0, address(0), false);
    }

    isValid = !cert.isRevoked;

    return (

```

```

        isValid,
        cert.studentId,
        cert.certType,
        cert.issuedAt,
        cert.issuer,
        cert.isRevoked
    );
}

/**
 * @dev Mencabut sertifikat. Hanya ADMIN atau KEPALA_SEKOLAH.
 */
function revokeCertificate(bytes32 documentHash, string memory reason) external nonReentrant {
    require(
        hasRole(KEPALA_SEKOLAH_ROLE, msg.sender) || hasRole(DEFAULT_ADMIN_ROLE, msg.sender),
        "CertificateRegistry: Must have KEPALA_SEKOLAH_ROLE or ADMIN_ROLE to revoke"
    );
    require(certificates[documentHash].issuedAt != 0, "CertificateRegistry: Certificate does not exist");
    require(!certificates[documentHash].isRevoked, "CertificateRegistry: Certificate already revoked");

    certificates[documentHash].isRevoked = true;
    certificates[documentHash].revokeReason = reason;

    emit CertificateRevoked(documentHash, reason, msg.sender, block.timestamp);
}

/**
 * @dev Mengambil semua hash sertifikat milik seorang siswa.
 */
function getCertificatesByStudent(string memory studentId) external view returns (bytes32[] memory) {
    return studentCertificates[studentId];
}

function getCertificateCount() external view returns (uint256) {
    return totalCertificates;
}

function getAllCertificateHashes() external view returns (bytes32[] memory) {
    return allCertificateHashes;
}

function grantKepalaSekolahRole(address account) external {
    require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender), "CertificateRegistry: Must have ADMIN_ROLE to grant roles");
    grantRole(KEPALA_SEKOLAH_ROLE, account);
}

function grantTURole(address account) external {
    require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender), "CertificateRegistry: Must have ADMIN_ROLE to grant roles");
    grantRole(TU_ROLE, account);
}
}

```

Kode 33. Smart Contract Audit Nilai (blockchain/contracts/GradeAuditLog.sol)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/access/AccessControl.sol";

```

```

/**
 * @title GradeAuditLog
 * @dev Smart contract untuk mencatat setiap perubahan nilai siswa secara immutable.
 * Mencegah kecurangan perubahan nilai oleh oknum internal.
 * Setiap perubahan dicatat di ledger blockchain dan tidak bisa dihapus.
 */
contract GradeAuditLog is AccessControl {
    bytes32 public constant GURU_ROLE = keccak256("GURU_ROLE");

    struct GradeChange {
        string studentId;
        string subjectId;
        uint256 oldScore;        // Score * 100 untuk handle desimal (e.g., 85.5 = 8550)
        uint256 newScore;
        string teacherId;
        uint256 timestamp;
        string reason;
    }

    mapping(string => GradeChange[]) public studentGradeHistory;
    GradeChange[] public allChanges;
    uint256 public totalChanges;

    event GradeChanged(
        string indexed studentId,
        string subjectId,
        uint256 oldScore,
        uint256 newScore,
        string teacherId,
        uint256 timestamp
    );

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(GURU_ROLE, msg.sender);
    }

    /**
     * @dev Mencatat perubahan nilai ke blockchain. Hanya GURU atau ADMIN.
     * @param studentId ID siswa
     * @param subjectId ID mata pelajaran
     * @param oldScore Nilai lama (dikali 100)
     * @param newScore Nilai baru (dikali 100)
     * @param teacherId ID guru yang mengubah
     * @param reason Alasan perubahan nilai
     */
    function logGradeChange(
        string memory studentId,
        string memory subjectId,
        uint256 oldScore,
        uint256 newScore,
        string memory teacherId,
        string memory reason
    ) external {
        require(
            hasRole(GURU_ROLE, msg.sender) || hasRole(DEFAULT_ADMIN_ROLE, msg.sender),
            "GradeAuditLog: Must have GURU_ROLE or ADMIN_ROLE to log grades"
        );

        GradeChange memory newChange = GradeChange({
            studentId: studentId,
            subjectId: subjectId,

```

```

        oldScore: oldScore,
        newScore: newScore,
        teacherId: teacherId,
        timestamp: block.timestamp,
        reason: reason
    });

    studentGradeHistory[studentId].push(newChange);
    allChanges.push(newChange);
    totalChanges++;

    emit GradeChanged(studentId, subjectId, oldScore, newScore, teacherId, block.timestamp);
}

/**
 * @dev Mengambil riwayat perubahan nilai untuk seorang siswa.
 */
function getStudentGradeHistory(string memory studentId) external view returns (GradeChange[] memory)
{
    return studentGradeHistory[studentId];
}

function getTotalChanges() external view returns (uint256) {
    return totalChanges;
}

function getChangeByIndex(uint256 index) external view returns (GradeChange memory) {
    require(index < allChanges.length, "GradeAuditLog: Index out of bounds");
    return allChanges[index];
}
}

```

Kode 34. Script Deployment (blockchain/scripts/deploy.js)

```

const hre = require("hardhat");
const fs = require("fs");
const path = require("path");

async function main() {
    console.log("=".repeat(60));
    console.log("SMK Cloud Security - Smart Contract Deployment");
    console.log("=".repeat(60));

    const [deployer] = await hre.ethers.getSigners();
    console.log("\nDeploying contracts with account:", deployer.address);
    console.log("Account balance:", (await hre.ethers.provider.getBalance(deployer.address)).toString());

    // Deploy CertificateRegistry
    console.log("\n[1/2] Deploying CertificateRegistry...");
    const CertificateRegistry = await hre.ethers.getContractFactory("CertificateRegistry");
    const certificateRegistry = await CertificateRegistry.deploy();
    await certificateRegistry.waitForDeployment();
    const certRegistryAddress = await certificateRegistry.getAddress();
    console.log(" ✅ CertificateRegistry deployed to:", certRegistryAddress);

    // Deploy GradeAuditLog
    console.log("\n[2/2] Deploying GradeAuditLog...");
    const GradeAuditLog = await hre.ethers.getContractFactory("GradeAuditLog");
    const gradeAuditLog = await GradeAuditLog.deploy();
    await gradeAuditLog.waitForDeployment();
    const gradeAuditLogAddress = await gradeAuditLog.getAddress();
    console.log(" ✅ GradeAuditLog deployed to:", gradeAuditLogAddress);
}

```

```

// Save addresses to JSON
const addresses = {
  CertificateRegistry: certRegistryAddress,
  GradeAuditLog: gradeAuditLogAddress,
  deployer: deployer.address,
  network: hre.network.name,
  deployedAt: new Date().toISOString()
};

const filePath = path.join(__dirname, "../deployed-addresses.json");
fs.writeFileSync(filePath, JSON.stringify(addresses, null, 2));

console.log("\n" + "=".repeat(60));
console.log("Deployment Complete!");
console.log("Deployed addresses saved to:", filePath);
console.log("=".repeat(60));
console.log("\nContract Addresses:");
console.log("  CertificateRegistry:", certRegistryAddress);
console.log("  GradeAuditLog:      ", gradeAuditLogAddress);
console.log("\nNext steps:");
console.log("  1. Update CONTRACT_ADDRESS in backend/.env");
console.log("  2. Start the backend server: cd ../backend && npm run dev");
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});

```

Kode 35. Hardhat Configuration (blockchain/hardhat.config.js)

```

require('@nomicfoundation/hardhat-toolbox');

module.exports = {
  solidity: {
    version: '0.8.24',
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  networks: {
    localhost: {
      url: 'http://127.0.0.1:8545'
    },
    hardhat: {
      chainId: 31337
    }
  }
};

```
